

# Agentic Cloud Modernization Platform: Architectural Blueprint v3.0

## Table of Contents

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Architectural Rationale Summary.....                                         | 4  |
| 1. Introduction and Strategic Vision .....                                   | 4  |
| 1.1. The Problem Statement .....                                             | 4  |
| 1.2. The Vision.....                                                         | 5  |
| 1.3. Scope.....                                                              | 5  |
| 2. Architectural Principles.....                                             | 5  |
| 2.1. Design Assurance Traceability Matrix.....                               | 6  |
| 3. High-Level Architecture .....                                             | 7  |
| 3.1. System Context Diagram .....                                            | 7  |
| 3.2. Detailed Architecture Diagram .....                                     | 8  |
| 4. Core Capabilities: Service Boundaries and Specialist Agent Inventory..... | 9  |
| 4.1. Service Boundaries .....                                                | 10 |
| 4.2. Specialist Agent Inventory.....                                         | 10 |
| 4.3. User Experience (UX) Vision: From Data to Decision .....                | 12 |
| 4.3.1. The AI Management & Collaboration Hub .....                           | 12 |
| 4.3.2. Core UX Principles for Building Trust and Usability .....             | 13 |
| 5. Detailed Architecture & Layered Technology Stack.....                     | 14 |
| 5.1. The Agentic Technology Stack .....                                      | 15 |
| 5.2. Core Platform & Infrastructure Stack (Google Cloud) .....               | 16 |
| 5.2.b. Cross-Cloud Deployment Mapping.....                                   | 17 |
| 5.3. High-Level User Experience Architecture .....                           | 19 |
| 5.3.b. WebSocket Authentication.....                                         | 20 |
| 5.4. Runtime Interaction Model: The Hierarchy of Control .....               | 21 |
| 5.5 Tool Registration and Certification Workflow .....                       | 22 |
| 5.6. Integration Pattern for Third-Party Agents (e.g., AWS-native) .....     | 23 |

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| 6. Communication Protocols and Data Schemas .....                                | 24 |
| 7. Step-by-Step Data Flow: Building Stage 1 Foundational Knowledge.....          | 24 |
| Phase 0: Engagement Setup & Discovery Initiation .....                           | 25 |
| Phase A — Collection & Evidence Ingestion .....                                  | 27 |
| Phase B — The Document Processing & Fusion Pipeline .....                        | 28 |
| Detailed Process Flow .....                                                      | 32 |
| 8. Step-by-Step Workflow: Stage 2 Insights Synthesis .....                       | 34 |
| Phase C — Structured Workflow Execution with CrewAI .....                        | 34 |
| Phase D — Interactive Conversational Assistance with AutoGen .....               | 37 |
| Phase E — Durable Execution with Temporal .....                                  | 38 |
| Phase F — Continuous Learning & Improvement.....                                 | 39 |
| Phase F - 1.b: The Human Correction & Feedback Workflow .....                    | 39 |
| 9. Final Architectural Considerations.....                                       | 40 |
| 9.1. Infrastructure & Network Security.....                                      | 40 |
| 9.1.b. Hybrid & On-Premises Deployment Topology.....                             | 42 |
| 9.2. Identity, Access, and Supply Chain Security .....                           | 43 |
| 9.3 Prompt Lifecycle Management .....                                            | 44 |
| 9.4 Scalability of the Knowledge Fusion Process .....                            | 45 |
| 9.5. Key Performance Indicators (KPIs) and Service Level Objectives (SLOs) ..... | 46 |
| 9.5.b. Performance Envelope & Cost Targets.....                                  | 46 |
| 9.6 AI Safety & Security Layer .....                                             | 47 |
| 9.6.b. Tenant Isolation and Prompt Sandboxing.....                               | 48 |
| 9.6.c. PII Handling, Deletion, and Secure Viewing .....                          | 49 |
| 9.7. Model Serving and Lifecycle Management.....                                 | 50 |
| 10. Platform CI/CD and Operational Model .....                                   | 51 |
| 10.1. The Polyrepo Structure.....                                                | 51 |
| 10.2. Automated CI/CD Pipelines .....                                            | 51 |
| 10.2.b. Federated CI/CD Workflow for Client-Managed Repositories .....           | 52 |
| 10.3. Operational Personas .....                                                 | 53 |

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 11. Performance, Scaling, and Cost Considerations .....            | 53 |
| 11.1. Performance & Scaling Strategy .....                         | 53 |
| 11.2. Cost Governance and Optimization .....                       | 54 |
| 11.3 Unified Observability Context.....                            | 55 |
| 12. Phased Rollout & Immediate Next Steps.....                     | 56 |
| The MVP: Functional Cut Line .....                                 | 57 |
| 13. Governance Model for the Reference Decision Library (RDL)..... | 58 |
| 13.1. Principles of RDL Governance .....                           | 59 |
| 13.2. RDL Governance Roles & Responsibilities .....                | 60 |
| 13.3. The RDL Change Management Workflow .....                     | 60 |
| 13.4. Knowledge Schema Versioning and Migration Strategy .....     | 61 |
| 14. Operational Mode: Continuous Advisory .....                    | 62 |
| 14.1. Live Data Adapters .....                                     | 62 |
| 15.1. Incident Categories & Automated Containment.....             | 63 |
| 15.2. Forensic & Recovery Process.....                             | 63 |
| 15.3. Failure Mode Examples & Recovery Stories .....               | 64 |
| 16. Appendices.....                                                | 65 |
| 16.1. Core Event Schema Examples (JSON Schema) .....               | 65 |
| 16.2. Minimum Audit Log Schema.....                                | 65 |
| 16.3. RDL & Prompt Versioning Strategy .....                       | 66 |
| 16.4. IaC Generation & Pre-Merge Validation .....                  | 66 |
| 16.5. Measurable MVP Acceptance Criteria .....                     | 67 |
| 16.6. MCP Tool & Token Schema Examples .....                       | 67 |
| 16.7 Discovery Permissions and Service Identities .....            | 68 |
| 17. Architect's Index.....                                         | 69 |
| 18. Future Considerations: Federated Control Plane.....            | 70 |

# Architectural Rationale Summary

This document details the architecture for the Agentic Modernization Platform, an intelligent co-pilot designed to accelerate enterprise cloud transformations. This summary provides the core rationale for its design, tailored for executive and board-level review.

- **Why This Platform Now? (Business Urgency):** The primary bottleneck in cloud adoption is no longer technology but the scarcity of senior architectural talent. This platform acts as a force multiplier, automating up to 80% of the repetitive discovery, analysis, and code generation tasks. This directly addresses the market gap by enabling a smaller team of experts to deliver faster, more consistent, and higher-quality migration outcomes, reducing both time-to-value and project risk.
- **Why This Tech Stack? (Strategic Choices):** Our technology choices prioritize enterprise-grade reliability and AI-native flexibility. We explicitly rejected complex big data frameworks like Spark for data fusion in favor of a durable workflow engine (**Temporal**). This provides superior deterministic reliability, error handling, and observability with significantly lower operational overhead. Our agentic layer combines the deterministic workflows of **CrewAI** with the conversational power of **AutoGen**, using **Semantic Kernel** as a universal adapter, representing the best-in-class layered stack for building complex AI systems.
- **Why This Governance Model? (Building Trust):** The platform's credibility rests on its governance. We are implementing a "trust but verify" model where all critical intellectual property—the **Reference Decision Library (RDL)**, AI Prompts, and Knowledge Graph Schema—are managed as code in version-controlled, governed repositories. Every change is subject to human review, automated impact analysis, and an auditable approval process, ensuring the platform's reasoning remains transparent and aligned with our organization's best practices.

---

## 1. Introduction and Strategic Vision

### 1.1. The Problem Statement

Enterprise cloud adoption and modernization initiatives are critical for business agility but are consistently hampered by significant challenges. The process of assessing legacy estates, planning migrations, and executing them securely is manual, time-consuming,

and heavily reliant on a small pool of senior architects. This leads to slow delivery, inconsistent outcomes, high costs, and a substantial risk of human error. Key pain points include incomplete discovery, subjective decision-making, inconsistent IaC quality, and a poor feedback loop for continuous improvement.

## 1.2. The Vision

We will build an **Agentic Modernization Platform**—an intelligent, AI-augmented system that acts as a co-pilot for our cloud architects. The platform will automate the most laborious aspects of discovery, analysis, and solution design, freeing up our experts to focus on high-value strategic decisions.

Our vision is to create a **continuously learning, auditable, and secure system that synthesizes foundational enterprise knowledge into well-architected cloud solutions with unparalleled speed and confidence.**

## 1.3. Scope

The platform is designed as an **any-to-any** modernization tool, capable of assessing workloads from on-premises (e.g., VMware) or any cloud provider (e.g., AWS, Azure) and planning their migration to any target cloud. The initial implementation will focus on virtualized server workloads, but the architecture is intentionally designed to be extensible to support broader modernization use cases, including application containerization, PaaS adoption, and FinOps-driven optimization.

---

# 2. Architectural Principles

These core principles guide all subsequent design decisions and technology choices.

1. **AI-Augmented, Architect-Driven:** The platform is a powerful co-pilot, not an autopilot. It automates data gathering and generates high-quality, evidence-backed recommendations through collaborative AI agents. The human architect retains ultimate authority, providing oversight, critical judgment, and essential feedback for the system's continuous learning.
2. **Two-Stage Knowledge Model:** The platform operates on a two-stage knowledge architecture.

1. **Stage 1** focuses on creating **Foundational Facts**—a canonical, deduplicated, and cross-validated knowledge graph extracted from source documents and discovery tools.
2. **Stage 2** focuses on **Insights Synthesis**, where intelligent agents reason over these foundational facts to generate higher-level analysis, plans, and recommendations.
3. **Auditability and Explainability First:** Every decision, proposal, and piece of generated code must be fully traceable back to the source evidence and the specific logic that produced it. The reasoning of the LLM must be transparent and verifiable through the RAG context and the conversational history of the agent crews.

**Secure by Design:** The platform operates in a zero-trust environment. Security is not an afterthought; it is built into every layer, with least privilege access, end-to-end encryption, robust data sanitization, and strict network controls. The structure and content of this architectural blueprint adhere to the recommendations outlined in the ISO/IEC 42010 standard for architecture description.

4. **Modular and Extensible Fabric:** The agent-based architecture is fundamental. It allows for independent development, scaling, and replacement of capabilities. The platform will leverage a **Model Context Protocol (MCP)** to allow for the dynamic, runtime discovery and loading of new tools and capabilities, ensuring future-proof extensibility without requiring redeployments.
5. **Cloud-Agnostic Logic, Cloud-Native Implementation:** The core decision-making logic, agentic skills, and data models are designed to be abstract and portable. However, the platform's implementation will aggressively leverage cloud-native managed services (e.g., Serverless, Managed Databases, Durable Workflows) to maximize reliability and scalability while minimizing operational overhead.

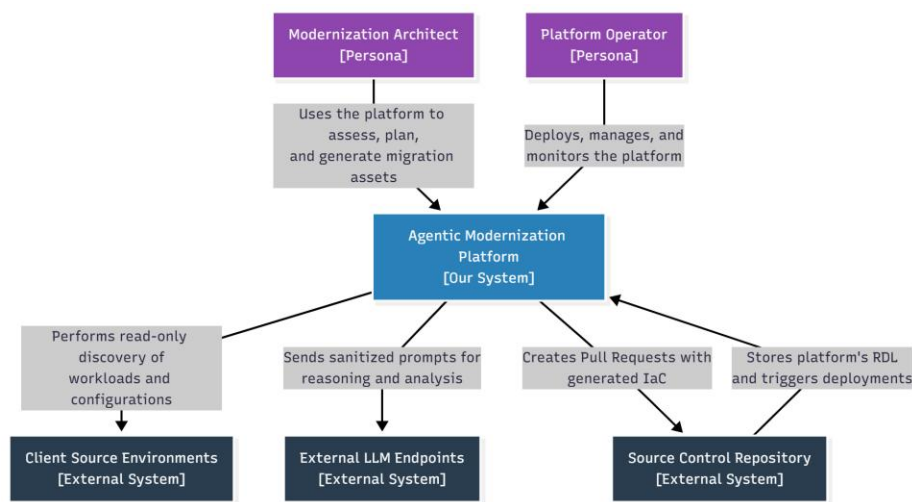
## 2.1. Design Assurance Traceability Matrix

The "Auditability and Explainability First" principle is enforced through specific design choices and verification mechanisms. The following matrix links key assurance areas to their implementation within this document and the platform.

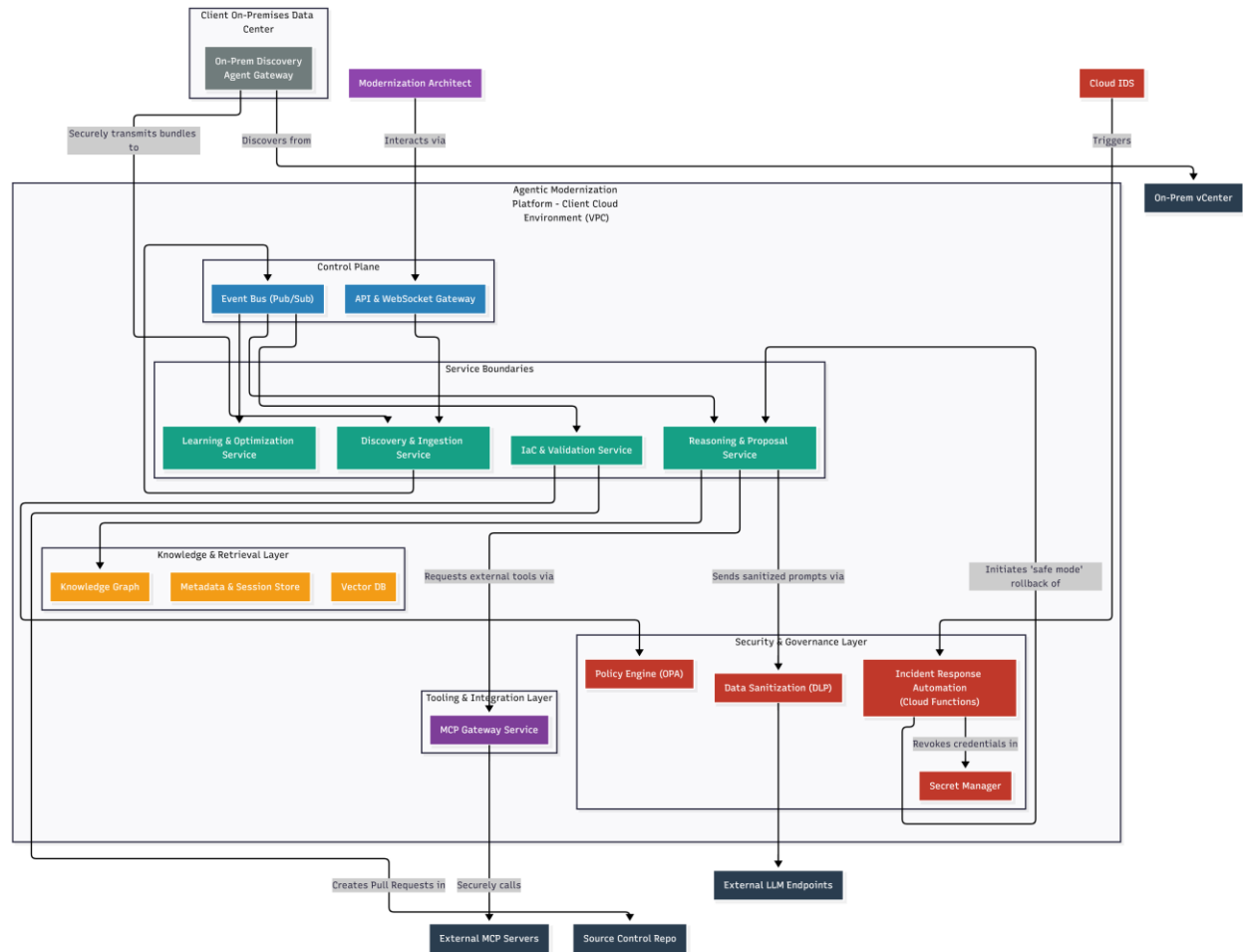
| ASSURANCE AREA   | RELEVANT DESIGN SECTION                         | VERIFICATION MECHANISM                                                                  |
|------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------|
| DATA LINEAGE     | 7.0 (The Document Processing & Fusion Pipeline) | <b>GCS EvidenceBundle Checksum</b> and Provenance links in the Knowledge Graph.         |
| PROMPT INTEGRITY | 9.3 (Prompt Lifecycle Management)               | <b>CI/CD Linting &amp; Golden Dataset Evaluation</b> for the prompt-library repository. |
| BUILD INTEGRITY  | 9.2 (Supply Chain Security with SLSA)           | <b>Signed SLSA Provenance</b> and the <b>Client-Facing Verification API/Runbook</b> .   |
| IAC SAFETY       | 18.4 (IaC Generation & Pre-Merge Validation)    | <b>OPA Validation</b> and the mandatory <b>Automated Sandbox Dry-Run</b> .              |
| MODEL DRIFT      | 9.7 (Model Serving and Lifecycle Management)    | <b>Golden Dataset Evaluation</b> as part of the model approval workflow.                |

### 3. High-Level Architecture

#### 3.1. System Context Diagram



## 3.2. Detailed Architecture Diagram



The platform is decomposed into four primary logical areas, all running within a secure, isolated client-hosted environment:

### 1. Agentic Fabric (Compute & Reasoning Layer):

- This is not a collection of monolithic services, but a sophisticated runtime for hosting crews of specialized, collaborative AI agents. It leverages a layered stack of agent frameworks to perform its tasks.
- *Technology: AutoGen for conversational reasoning, CrewAI for deterministic workflows, Semantic Kernel for tool orchestration, and Temporal for long-running, durable execution.*

### 2. Thin Control Plane (Orchestration & API Layer):



- Provides the central API gateway, user interface backend, and the event bus for asynchronous communication between the major service domains.
- *Technology: FastAPI, API Gateway + IAP, Pub/Sub (with A2A-compliant schemas), WebSockets.*

### 3. Knowledge & Retrieval Layer (State & Memory Layer):

- The platform's comprehensive memory, representing the **Stage 1 Foundational Facts**. It's a collection of specialized data stores for structured metadata, graph relationships, vector embeddings, and raw evidence.
- *Technology: Cloud SQL (Postgres), Neo4j, Vertex AI Matching Engine, Elasticsearch, Google Cloud Storage.*

### 4. Security & Governance Layer (Cross-Cutting Concern):

- Provides foundational security services to all other layers, including secret management, encryption key management, data sanitization, and a detailed audit trail.
- *Technology: Secret Manager, Cloud KMS (CMEK), Cloud DLP, Cloud Logging, Open Policy Agent.*

### 5. Tooling & Integration Layer (Extensibility Fabric):

- Provides a secure, managed gateway for agents to dynamically discover and consume tools from external systems. This is the primary mechanism for interacting with live cloud APIs and third-party services.
- *Technology: Model Context Protocol (MCP) Gateway Service, custom-built tool adapters.*

---

## 4. Core Capabilities: Service Boundaries and Specialist Agent Inventory

We organize the platform's capabilities into logical **Service Boundaries**, which represent the major functional domains and deployable microservices. The intelligent work *within* these boundaries is performed by a rich inventory of specialized, fine-grained **Specialist Agents** that are composed into crews. This section defines both layers of our architecture.

---

## 4.1. Service Boundaries

These are the high-level, containerized services that make up the platform's backend. They correspond to the major phases of the transformation lifecycle.

- **Discovery & Ingestion Service (The Document Processor):**
  - **Responsibility:** This service is responsible for all of **Stage 1: Building Foundational Knowledge**. It connects to client sources via discovery adapters, runs the document processing pipeline, performs OCR, and orchestrates the entity fusion process to build and maintain the canonical knowledge graph.
- **Reasoning & Proposal Service (The AI Core):**
  - **Responsibility:** This service is the heart of **Stage 2: Insights Synthesis**. It hosts the AutoGen and CrewAI runtimes. Its job is to assemble agent crews, execute complex reasoning workflows to generate proposals, and handle interactive, conversational "what-if" analysis from the user.
- **Generation & Execution Service (The Delivery Engine):**
  - **Responsibility:** This service is focused on turning approved plans into reality. It is responsible for converting architect-approved proposals into high-quality IaC, creating pull requests, validating the code in an automated sandbox, and orchestrating the long-running Temporal workflows for deployment and post-flight validation.
- **Learning & Optimization Service (The Improvement Engine):**
  - **Responsibility:** This service is responsible for the platform's continuous learning and optimization loops. It analyzes all feedback signals (e.g., ProposalCorrected events, post-deployment metric deltas), proposes automated updates to the RDL, triggers model retraining pipelines, and performs continuous FinOps cost analysis

## 4.2. Specialist Agent Inventory

These are the fine-grained, role-based AI personas that perform the actual work. They are the single source of truth for agent capabilities and are instantiated and managed by the **Reasoning & Proposal Service**.

| Agent Canonical Name      | Role / Title                                     | Framework       | Primary Function                                                                                                        |
|---------------------------|--------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| engagement_analyst        | Senior Infrastructure Discovery Analyst          | CrewAI          | Performs cross-modal synthesis to build the initial Project Context, bridging Stage 1 facts and Stage 2 insights.       |
| principal_cloud_architect | Principal Cloud Architect & Migration Strategist | CrewAI, AutoGen | Designs target cloud architectures, analyzes migration patterns, and makes modernization recommendations.               |
| risk_compliance_officer   | Risk & Compliance Officer                        | CrewAI          | Audits proposed architectures against security and compliance frameworks (GDPR, SOC2, HIPAA).                           |
| lead_planning_manager     | Lead Migration Program Manager                   | CrewAI          | Synthesizes technical and risk analysis into executive-ready migration plans and timelines.                             |
| document_researcher       | Document Research Specialist                     | CrewAI          | Gathers and synthesizes comprehensive information from the knowledge base for a specific document type.                 |
| content_architect         | Content Architecture Specialist                  | CrewAI          | Structures and organizes the final output of any workflow into a professional, human-readable document.                 |
| quality_reviewer          | Document Quality Assurance Specialist (Critic)   | CrewAI          | Critically evaluates the work of other agents for accuracy, completeness, and logical consistency.                      |
| post_processing_agent     | Lessons Learned Analyst                          | CrewAI          | Analyzes the outcomes of completed workflows to synthesize new best practices for the RDL.                              |
| devops_expert             | DevOps Expert                                    | AutoGen         | Provides deep expertise on CI/CD, IaC best practices, containerization, and SRE principles.                             |
| cost_optimizer            | FinOps Expert                                    | AutoGen         | Analyzes financial implications of architectural decisions and performs rightsizing analysis.                           |
| data_expert               | Data Expert                                      | AutoGen         | Specializes in database migration, data lake architecture, and ETL/ELT pipeline design.                                 |
| app_modernization_expert  | App Modernization Expert                         | AutoGen         | Focuses on strategies for refactoring legacy applications and microservices.                                            |
| modeling_agent            | Modeling Agent                                   | CrewAI, AutoGen | Specialist in top-down, assumption-driven reasoning to create artifacts like business cases from high-level parameters. |
| organizational_analyst    | Organizational Analyst                           | CrewAI          | Analyzes human-centric data to perform skills gap analysis and identify key stakeholders.                               |
| change_management_agent   | Change Management Agent                          | CrewAI          | Produces tailored change management and communications plans for impacted teams.                                        |
| guardrail_agent           | Guardrail Agent                                  | Advisory        | Continuously monitors live cloud spend against budgets and triggers real-time anomaly alerts.                           |
| sre_agent                 | SRE Agent                                        | Advisory        | Continuously monitors performance dashboards and suggests reliability improvements.                                     |

|                         |                         |          |                                                                       |
|-------------------------|-------------------------|----------|-----------------------------------------------------------------------|
| maturity_assessor_agent | Maturity Assessor Agent | Advisory | Runs on a schedule to re-assess cloud maturity and update scorecards. |
|-------------------------|-------------------------|----------|-----------------------------------------------------------------------|

### 4.3. User Experience (UX) Vision: From Data to Decision

The primary user of this platform is a highly skilled but time-constrained Modernization Architect. The UX vision is to create an intuitive **co-pilot experience** that empowers, rather than overwhelms, them with data. The UI is the primary surface for collaboration, allowing the architect to direct, manage, and trust the AI's work. This is achieved through a set of dedicated management interfaces and three core UX principles.

#### 4.3.1. The AI Management & Collaboration Hub

To provide the necessary transparency and control, the UI will feature a centralized hub for managing the platform's agentic capabilities. This hub is designed for architects and platform operators to understand and guide the AI, not to code it. It consists of:

- **The Deliverable Library:** This is the architect's starting point for generating high-value, structured documents. It is a browsable catalog of pre-configured "Global Document Templates" (e.g., *Cloud Readiness Scorecard*, *Total Cost of Ownership Analysis*). Selecting a template launches a pre-defined CrewAI workflow, providing a simple, one-click interface to initiate complex, multi-agent tasks.
- **The AI Crew Manager:** This interface provides deep transparency into how deliverables are created. It allows users to:
  - **View & Inspect:** See the full inventory of available Agents, the Tasks they can perform, and how they are composed into Crews.
  - **Configure & Tune:** For platform operators or expert users, this interface provides the ability to tune agent behavior by modifying parameters such as their role or backstory. This allows the platform to be tailored to specific client contexts or engagement needs without changing any underlying code.
- **The Agent & Tool Registry:** A read-only, browsable catalog of all available specialist agents and the tools they are equipped with. This serves as a clear reference for what the platform is capable of and how its agents can interact with data and external systems.

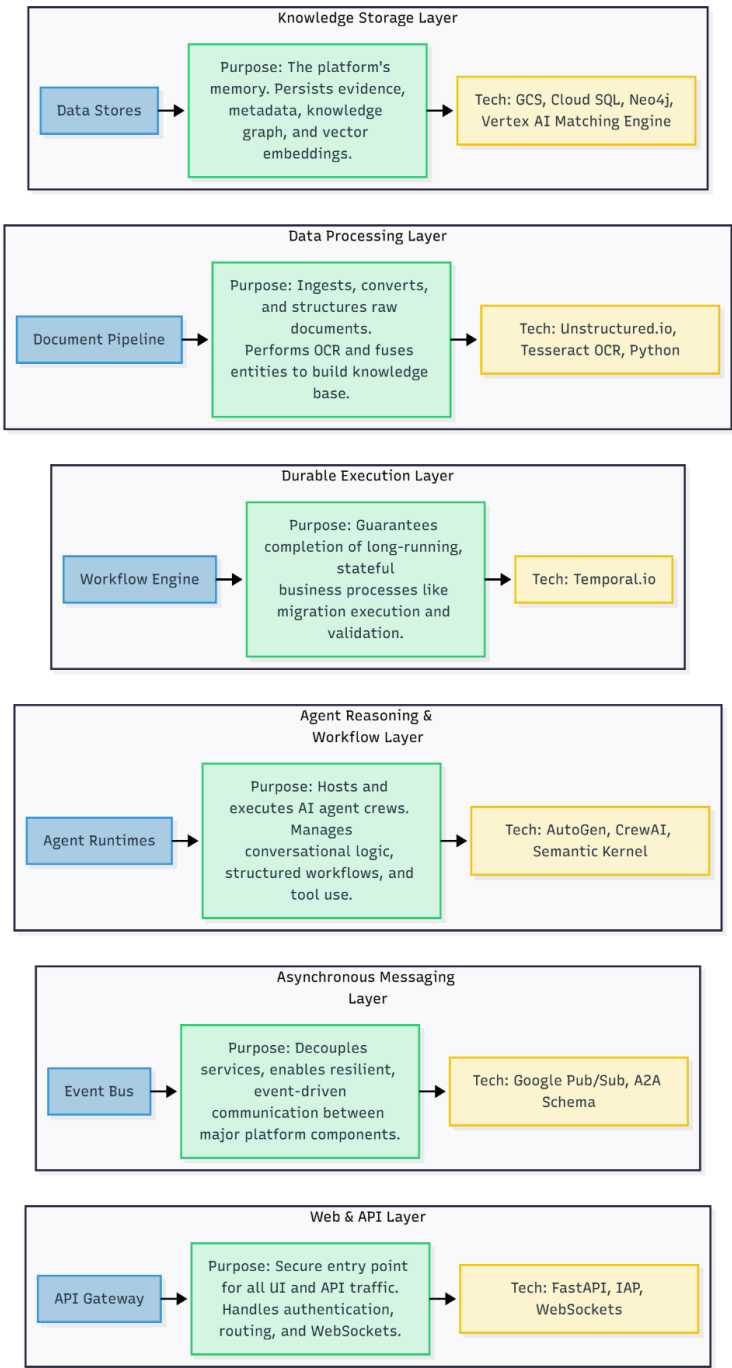
### 4.3.2. Core UX Principles for Building Trust and Usability

The following principles are foundational to the entire user experience and ensure that the platform is an effective and trustworthy co-pilot.

1. **Explainability and Trust through Traceability:** The user must be able to trust the AI's recommendations. The UI will not simply present a final answer; it will present a well-reasoned argument.
    - **Implementation:** Every piece of a proposal—every recommended instance type, every identified security risk—will be visually linked directly back to the source evidence. The user can click on a recommendation and immediately see a side panel showing the exact text from the source runbook, the specific metric from the discovery data, and the RDL rule that led to the conclusion. This "show your work" approach is fundamental to building user trust.
  2. **Interactive Simulation and Exploration:** The platform is not a static report generator; it is an interactive decision-support tool. The UI will be designed for exploration and "what-if" analysis.
    - **Implementation:** The proposal review screen will feature interactive controls. An architect can use a slider to change their risk tolerance, toggle between "Cost-Optimized" and "Performance-Optimized" modes, or manually override a parameter (as described in the Human Correction Workflow). The SimulateAgent (or an AutoGen crew) will then re-evaluate the proposal in real-time, instantly showing the impact of these changes on cost, compliance, and complexity. This turns the architect from a passive reviewer into an active participant in the design process.
  3. **Progressive Disclosure of Complexity:** An architect should not be flooded with every piece of data at once. The UI will use a "progressive disclosure" model to manage cognitive load.
    - **Implementation:** The initial view of a proposal will be a high-level executive summary with key findings and a single confidence score. The user can then drill down into specific areas of interest—clicking on "Networking" expands to show the proposed firewall rules and dependencies, clicking on "Cost" reveals the detailed cost breakdown. This layered approach allows the user to control the level of detail they engage with, making the vast amount of underlying data manageable.
-

# 5. Detailed Architecture & Layered Technology Stack

The platform's architecture is best understood as a multi-layered stack where each layer uses the best-in-class tool for a specific job, from reliable, long-running execution at the bottom to collaborative agentic reasoning at the top.



Component purpose and tech stack

## 5.1. The Agentic Technology Stack

| Layer                                                    | Purpose                                                                 | Primary Technology                                                                                       | Rationale                                                                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Layer 4:</b><br><i>Durable Workflow Engine</i>        | Reliability for long-running, stateful business processes (hours/days). | <b>Temporal</b>                                                                                          | Provides guaranteed execution, state checkpointing, retries, and observability for critical, multi-step operations like the migration and validation workflow. Essential for enterprise-grade resilience.                                                                                                                                         |
| <b>Layer 3:</b><br><i>Agent Orchestration Frameworks</i> | Collaborative reasoning and structured workflows (seconds/minutes).     | <b>AutoGen</b> (Conversational), <b>Crew AI</b> (Workflow), <b>Semantic Kernel</b> (Tool Orchestration)  | <b>AutoGen</b> enables dynamic, multi-agent conversations for interactive problem-solving. <b>CrewAI</b> provides the framework for defining deterministic, role-based workflows for tasks like document generation. <b>Semantic Kernel</b> acts as the universal adapter, allowing any agent to reliably plan, invoke tools, and manage prompts. |
| <b>Layer 2:</b><br><i>RAG &amp; Tooling Libraries</i>    | Knowledge access and action execution.                                  | <b>LlamaIndex &amp; LangChain</b>                                                                        | The workhorse libraries that provide the essential "plumbing" for the RAG pipeline. They offer pre-built connectors to data sources, document chunkers, and retrieval algorithms, forming the toolbelt that agents use to interact with the world.                                                                                                |
| <b>Layer 1:</b><br><i>Foundational Data Stores</i>       | The platform's persistent memory and canonical knowledge.               | <b>Google Cloud Services:</b> Cloud SQL (Postgres), Neo4j, Vertex AI Matching Engine, Elasticsearch, GCS | A curated set of specialized, managed data stores optimized for different data types—structured metadata, graph relationships, vector embeddings, keyword search, and raw object storage.                                                                                                                                                         |

|                          |                                           |                                                  |                                                                                                                                                                                                                                                     |
|--------------------------|-------------------------------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cross-Cutting Bus</b> | Asynchronous inter-service communication. | <b>Pub/Sub</b> with <b>A2A-compliant</b> schemas | Decouples the major service boundaries, providing resilience and scalability. Adopting an A2A-compliant schema for events ensures future interoperability with third-party agent systems.                                                           |
| <b>MCP Gateway</b>       |                                           | Custom Go/Python Service on Cloud Run            | Provides a single, secure, and governable entry point for agents to access external tools. It handles discovery, caching of tool definitions, and authentication to registered MCP servers, abstracting this complexity from the agents themselves. |
| <b>OCR Engine</b>        |                                           | Tesseract OCR                                    |                                                                                                                                                                                                                                                     |

## 5.2. Core Platform & Infrastructure Stack (Google Cloud)

| <b>Component</b>       | <b>Chosen Technology</b>                         | <b>Rationale</b>                                                                                                                                             |
|------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API Framework</b>   | <b>FastAPI</b>                                   | High-performance, async framework with automatic OpenAPI documentation, ideal for building the platform's REST APIs.                                         |
| <b>Compute Runtime</b> | <b>Cloud Run</b> (Primary), <b>GKE Autopilot</b> | <b>Cloud Run</b> for scalable, serverless containers. <b>GKE Autopilot</b> for services requiring specialized hardware (GPUs) or complex stateful workloads. |
| <b>API Gateway</b>     | <b>API Gateway + IAP</b>                         | Provides a managed, secure entry point for all UI and external API calls, with robust zero-trust authentication via IAP.                                     |
| <b>Real-time Comms</b> | <b>WebSockets</b>                                | Enables real-time, bidirectional streaming of agent conversations and progress notifications to the user interface.                                          |
| <b>OCR Engine</b>      | <b>Tesseract OCR</b>                             | Integrated via a containerized service to provide optical character recognition for scanned documents and images.                                            |



|                          |                                                  |                                                                                                                                  |
|--------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Security Services</b> | <b>Cloud DLP, Cloud KMS, Secret Manager</b>      | A suite of managed services providing data sanitization, customer-managed encryption keys (CMEK), and secure secret storage.     |
| <b>CI/CD &amp; IaC</b>   | <b>Terraform, Cloud Build, Artifact Registry</b> | The standard for infrastructure as code and serverless CI/CD, enabling automated, repeatable deployments of the platform itself. |
| <b>Observability</b>     | <b>OpenTelemetry + Cloud Operations Suite</b>    | Provides unified, end-to-end observability (metrics, logs, traces) across all services and agentic workflows.                    |

### 5.2.b. Cross-Cloud Deployment Mapping

While the primary implementation detailed above uses Google Cloud managed services, the platform's architecture is fundamentally cloud-agnostic. The "Cloud-Agnostic Logic, Cloud-Native Implementation" principle is realized by maintaining a consistent logical architecture while swapping the underlying cloud-native services.

The following table maps the key components of our GCP-based implementation to their direct equivalents in Amazon Web Services (AWS) and Microsoft Azure. This serves as the blueprint for deploying the platform in a multi-cloud or client-specific environment.

| <b>Architectural Component / Role</b> | <b>Primary Implementation (GCP)</b> | <b>AWS Equivalent</b>                    | <b>Azure Equivalent</b>               |
|---------------------------------------|-------------------------------------|------------------------------------------|---------------------------------------|
| <b>Serverless Compute</b>             | <b>Cloud Run</b>                    | <b>AWS Fargate</b>                       | <b>Azure Container Apps</b>           |
| <b>Kubernetes Compute</b>             | <b>GKE Autopilot</b>                | <b>Amazon EKS</b> (with Fargate profile) | <b>Azure Kubernetes Service (AKS)</b> |
| <b>API Gateway</b>                    | <b>API Gateway + IAP</b>            | <b>Amazon API Gateway + AWS WAF</b>      | <b>Azure API Management</b>           |

|                                |                                        |                                                          |                                                               |
|--------------------------------|----------------------------------------|----------------------------------------------------------|---------------------------------------------------------------|
| <b>Asynchronous Messaging</b>  | <b>Pub/Sub</b>                         | <b>Amazon SNS / SQS, EventBridge</b>                     | <b>Azure Service Bus, Event Grid</b>                          |
| <b>Durable Workflows</b>       | <b>Temporal</b> (Self-hosted on GKE)   | <b>Temporal</b> (Self-hosted on EKS), AWS Step Functions | <b>Temporal</b> (Self-hosted on AKS), Azure Durable Functions |
| <b>Relational Metadata</b>     | <b>Cloud SQL</b> (Postgres)            | <b>Amazon RDS</b> (for PostgreSQL)                       | <b>Azure Database for PostgreSQL</b>                          |
| <b>Knowledge Graph</b>         | <b>Neo4j</b> (AuraDB or self-hosted)   | <b>Amazon Neptune</b> , Neo4j (Aura or self-hosted)      | <b>Azure Cosmos DB</b> (with Gremlin API)                     |
| <b>Vector Search</b>           | <b>Vertex AI Matching Engine</b>       | <b>Amazon OpenSearch Service</b> (with k-NN), Pinecone   | <b>Azure AI Search</b> (with vector search)                   |
| <b>Keyword Search</b>          | <b>Elasticsearch</b> (Elastic Cloud)   | <b>Amazon OpenSearch Service</b>                         | <b>Azure AI Search</b>                                        |
| <b>Object/Evidence Store</b>   | <b>Google Cloud Storage (GCS)</b>      | <b>Amazon S3</b>                                         | <b>Azure Blob Storage</b>                                     |
| <b>Secret Management</b>       | <b>Secret Manager</b>                  | <b>AWS Secrets Manager</b>                               | <b>Azure Key Vault</b>                                        |
| <b>Encryption Keys</b>         | <b>Cloud KMS</b>                       | <b>AWS Key Management Service (KMS)</b>                  | <b>Azure Key Vault</b>                                        |
| <b>Data Sanitization (PII)</b> | <b>Cloud DLP</b>                       | <b>Amazon Macie</b>                                      | <b>Microsoft Purview</b>                                      |
| <b>CI/CD Platform</b>          | <b>Cloud Build</b> , Artifact Registry | <b>AWS CodePipeline/CodeBuild</b> , ECR                  | <b>Azure Pipelines</b> , ACR                                  |

|                      |                               |                                       |                                            |
|----------------------|-------------------------------|---------------------------------------|--------------------------------------------|
| <b>Observability</b> | <b>Cloud Operations Suite</b> | <b>Amazon CloudWatch, AWS X-Ray</b>   | <b>Azure Monitor</b> (incl. Log Analytics) |
| <b>LLM Endpoints</b> | <b>Vertex AI (Gemini)</b>     | <b>Amazon Bedrock</b> (Claude, Titan) | <b>Azure OpenAI Service</b> (GPT models)   |

### Implementation Strategy:

The platform's **Generation & Execution Service** is the key to managing this multi-cloud capability. It contains a library of cloud-specific Terraform modules. When generating Infrastructure as Code, it intelligently selects the appropriate module (e.g., aws-ec2-instance vs. gcp-compute-engine) based on the target cloud defined in the approved proposal. This ensures that the core agentic reasoning remains entirely separate from the cloud-specific implementation details.

## 5.3. High-Level User Experience Architecture

The interactive and real-time nature of the **Co-Pilot Experience** is powered by a modern, decoupled frontend architecture designed for responsiveness and scalability.

- **Frontend Framework:** The UI will be a **Single-Page Application (SPA)** built with a modern, component-based framework such as **React** or **Vue.js**. This approach allows for a fluid and dynamic user interface that can update in real-time without requiring page reloads.
- **Backend-for-Frontend (BFF) Pattern:** The frontend SPA does not communicate directly with the various backend microservices. Instead, it interacts exclusively with the **API & WebSocket Gateway**, which serves as its dedicated Backend-for-Frontend (BFF). This pattern provides several advantages:
  - **Simplified API:** The BFF exposes a single, tailored API endpoint for the frontend, aggregating data from multiple downstream services (Reasoning Service, Discovery Service, etc.) into a format optimized for the UI.
  - **Enhanced Security:** It provides a single point of authentication (via IAP) and authorization for all user-facing interactions.

- **Decoupling:** The frontend is completely decoupled from the internal microservice architecture, allowing the backend to be refactored or changed without impacting the user interface.
- **Asynchronous Job Correlation for Long-Running Tasks:** For long-running jobs, such as generating a comprehensive assessment document, the system uses a robust, non-blocking workflow to provide real-time feedback to the user without tying up resources.
  1. **Initiation:** The UI initiates the job via a standard REST API call to the API & WebSocket Gateway.
  2. **Job Acknowledgment:** The backend immediately acknowledges the request and returns a unique jobId. The HTTP connection is then closed.
  3. **Real-time Subscription:** The frontend UI immediately opens a **WebSocket** connection and subscribes to a specific channel or topic associated with that jobId (e.g., /ws/jobs/{jobId}).
  4. **Backend Event Propagation:** As the backend services (e.g., the CrewAI workflow) complete various stages of the task, they publish progress events to Pub/Sub.
  5. **Push Notification:** A dedicated notifier service listens to these Pub/Sub events and pushes targeted, real-time progress updates to the appropriate WebSocket channel.
  6. **UI Update:** The UI receives these push notifications and updates the progress bar or status message for the user, providing a seamless, real-time experience for long-running asynchronous operations.

### 5.3.b. WebSocket Authentication

WebSocket connections are a direct line into the backend and must be secured.

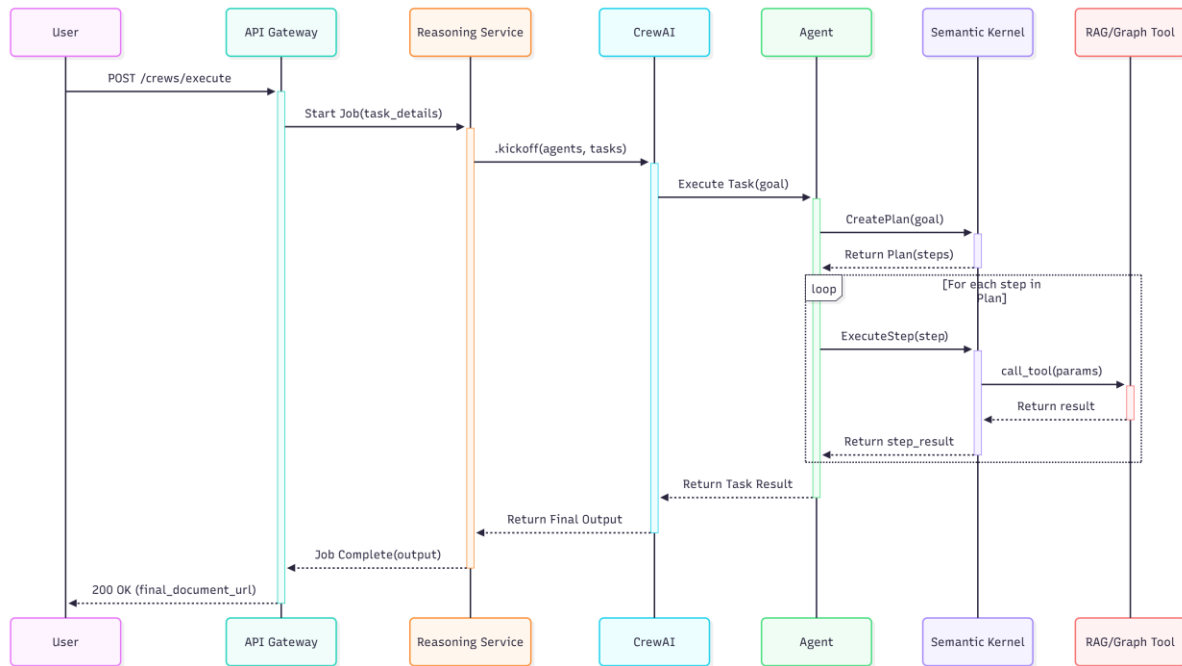
Authentication is handled during the initial connection upgrade request:

1. When the frontend SPA initiates the WebSocket connection, it includes the user's standard OAuth bearer token (obtained via IAP) in the Sec-WebSocket-Protocol header or as a query parameter.
2. The API & WebSocket Gateway intercepts this upgrade request, validates the token, and confirms the user's identity and permissions.

- Only upon successful validation is the connection upgraded to a WebSocket. The backend then associates the persistent connection with the validated user identity, ensuring all subsequent messages on that channel are properly authorized.

## 5.4. Runtime Interaction Model: The Hierarchy of Control

The agent frameworks are not peers; they operate in a clear hierarchy to manage complexity, context, and state.



- Control Flow:**

- CrewAI is the Master Orchestrator for Workflows:** For a given deliverable (e.g., "Generate Assessment"), a CrewAI crew is the top-level entry point. It defines the sequence of Tasks.
  - Tasks are Assigned to Agents:** CrewAI assigns a Task to a specialized Agent (e.g., the Principal Cloud Architect).
- Semantic Kernel is the Agent's "Brain":** The Agent does not call tools directly. It uses Semantic Kernel to plan its actions. This is a key architectural pattern. The agent provides its high-level goal (from the CrewAI task), and the SK Planner automatically discovers the necessary tools, chains them together, and

executes the plan. This decouples the agent's reasoning from the specifics of tool implementation and is a primary use case for Semantic Kernel.

- **AutoGen is for Interactive Sessions:** AutoGen operates in a separate, conversational mode for the chat interface. It acts as the master orchestrator for these sessions, but its agents still use Semantic Kernel to invoke their tools.
- **Context & State Management:**
  - **Job-Level State:** The Reasoning & Proposal Service holds the master state for a given job (e.g., job\_id, status).
  - **Agent State:** Agents are designed to be largely **stateless**. The context they need to perform a task is passed to them by CrewAI.
  - **Conversation State:** Long-term conversation history for AutoGen sessions is persisted in the **PostgreSQL** conversation\_messages table to provide memory between sessions.
  - **Error Recovery:** For the MVP, a failed task fails the entire CrewAI job, which is logged for review. In the Post-MVP phase, Temporal will orchestrate the CrewAI workflow, allowing for individual task retries and durable, long-running agentic processes.

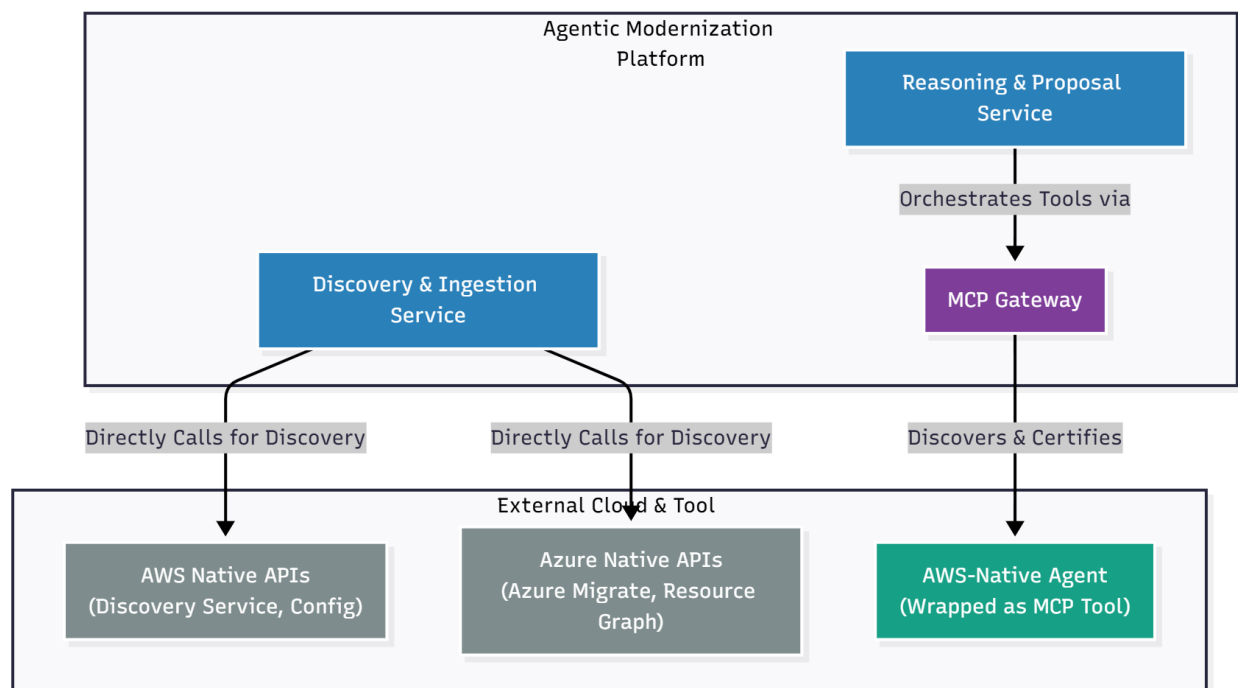
## 5.5 Tool Registration and Certification Workflow

To prevent agent sprawl and ensure that only secure, reliable, and validated tools are available to agents, the MCP Gateway Service includes a formal "Tool Certification" workflow. A tool is not available for use until it has passed this process.

- **1. Registration:** A developer (or Platform Operator) registers a new MCP server endpoint with the MCP Gateway. At this point, the tools are discovered but are in a "**quarantined**" state.
- **2. Manifest Validation:** The gateway ingests the Capability Manifest from each tool on the server. It performs a strict schema validation to ensure the manifest correctly defines its functions, parameters, and required security scopes. A manifest without a clear scope definition is automatically rejected.
- **3. Security & Functional Review:** A human operator (e.g., a member of the security team or RDL reviewer) must review the tool's capabilities and intended function. For high-risk tools (e.g., those that access sensitive data), a more thorough security review may be required.

- **4. Certification & Signing:** Upon approval, the Capability Manifest is cryptographically **signed** by the platform's authority. The tool is now moved from "quarantined" to "**certified**" and becomes visible to the Reasoning Service.
- **5. Runtime Enforcement:** The Reasoning Service is configured to only request capability tokens for tools that have a valid "certified" status in the MCP Gateway's registry. This ensures that no agent can ever be granted access to an unvetted or unapproved tool.

## 5.6. Integration Pattern for Third-Party Agents (e.g., AWS-native)



*Federated Multi-Cloud Agent Integration*

The platform is designed to integrate, not just replace, existing agentic capabilities, such as agents built natively for AWS. The **Model Context Protocol (MCP)** provides a standardized pattern for onboarding these external agents as tools without requiring them to be rewritten. The process is as follows:

1. **Interface Adaptation:** The external agent (e.g., an AWS agent that uses the AWS SDK) is wrapped in a lightweight service that exposes its functions as a REST API. This service acts as the MCP-compliant interface.

2. **Manifest Creation:** A Capability Manifest is created for this service, declaring its functions and required IAM scopes (e.g., aws:discovery:read).
  3. **Registration & Certification:** The new MCP endpoint is registered with the platform's MCP Gateway and passes through the standard **Tool Registration and Certification Workflow**.
  4. **Consumption:** Once certified, the external agent's capabilities become available as standard tools to the platform's internal agent crews, seamlessly integrating its functionality into our orchestrated workflows.
- 

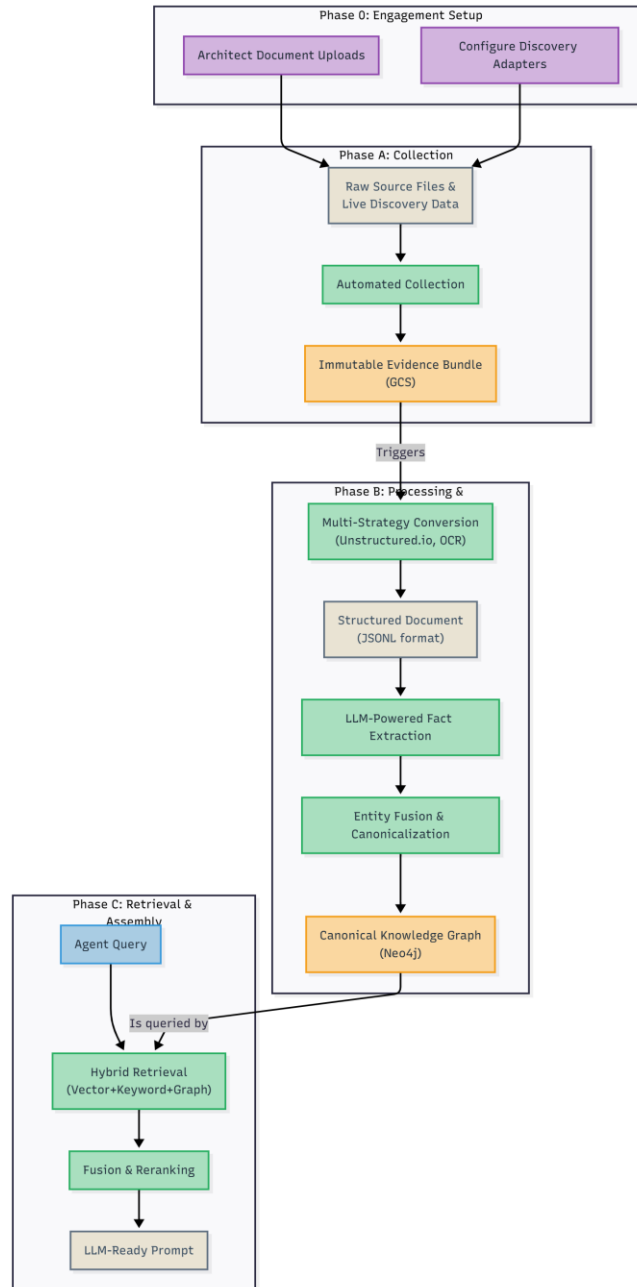
## 6. Communication Protocols and Data Schemas

- **Synchronous (API):** All user-facing and external API interactions are synchronous, using **REST/JSON over HTTPS** and secured by API Gateway + IAP.
  - **Asynchronous (Events):** Communication between the major service boundaries (Discovery, Reasoning, etc.) is asynchronous, using typed events published to **Pub/Sub**. This ensures resilience and loose coupling.
  - **Real-time (Streaming):** Interactive agent conversations and progress updates are streamed to the UI using **WebSockets**.
  - **Schema Management:** All event and API schemas will be defined in a version-controlled **JSON Schema** repository. This acts as the data contract for the entire system, with validation enforced at the service boundaries.
- 

## 7. Step-by-Step Data Flow: Building Stage 1 Foundational Knowledge

This section details the sophisticated, production-hardened pipeline for transforming raw, multi-format source data into a high-fidelity, canonical knowledge graph. This is the bedrock upon which all agentic reasoning is built.





*Data and context flow diagram*

## Phase 0: Engagement Setup & Discovery Initiation

Before the automated pipeline can begin, a **Modernization Architect** must initiate the engagement through the platform's UI. The platform supports two primary discovery modes, which can be used individually or in combination to build a comprehensive knowledge base.

- **Live Discovery Mode:** The preferred mode for ground-truth data. The architect configures Discovery Adapters for direct, read-only API access to the client's live environment (vCenter, AWS, etc.).
- **Offline/File-Based Discovery Mode:** For clients with strict security policies or air-gapped environments. The client provides data exports (e.g., CSV, XLS), which are then uploaded as documents.

### Step 1: Project Creation in the Management Portal

The architect begins by creating a new "Engagement Project" in the platform's UI. This creates an isolated logical workspace and a dedicated GCS bucket (gs://[project-id]-evidence/) for all subsequent artifacts.

### Step 2: Configuring Discovery Adapters (Live or Offline)

The architect chooses the appropriate discovery mode(s) for the engagement:

- **For Live Discovery:** The architect configures one or more of the agentless adapters via the UI.
  - **On-Premises (e.g., VMware vSphere):** *(The rest of the text for this remains the same).*
  - **Public Cloud (e.g., AWS, Azure, GCP):** Our adapters orchestrate the use of the respective cloud provider's native discovery services (e.g., AWS Migration Evaluator, Azure Migrate). As no single native tool is a silver bullet for "any-to-any" discovery, our adapters can also be configured to call the source cloud's inventory APIs directly to gather a complete asset list.
- **For Offline/File-Based Discovery:**
  - **Action:** The client is provided with a set of "Data Ingestion Templates" (e.g., pre-formatted Excel sheets) to fill out with their exported data (e.g., from RVTools or a CMDB). The architect then uploads these completed files as part of the standard "Uploading Existing Documentation" step. This provides a structured path for ingestion when direct API access is not permitted.

### Step 3: Uploading Existing Documentation (Documentary Discovery)

This step is critical for capturing the business context, architectural decisions, and operational knowledge that live discovery cannot see.

- **Action:** The architect uses the UI to perform a bulk upload of relevant existing documentation into a dedicated uploads\_raw/ prefix in the project's GCS bucket.
- **Examples of Documents:**

- Architectural diagrams (Visio, Draw.io)
- Runbooks and operational guides (Word, PDF)
- Application dependency spreadsheets (Excel)
- CMDB exports (CSV)
- Security compliance reports
- Business continuity plans

#### **Step 4: Ingesting Qualitative & Strategic Data**

The platform must reason over more than just technical data. This step allows architects to provide the human-centric and strategic context necessary for organizational and strategy-focused deliverables.

- **Action:** The architect uses dedicated UI forms or document uploads to provide qualitative data.
- **Examples of Data:**
  - Transcripts from stakeholder interviews.
  - Results from organizational readiness surveys (e.g., from SurveyMonkey).
  - HR system exports containing roles and skills data.
  - High-level strategic documents detailing Business Drivers and OKRs.

Once both the live discovery adapters have been configured and the existing documentation has been uploaded, the architect can officially "Start Ingestion." This action triggers the publication of the initial DiscoveryEvents, which formally kicks off **Phase A** of the automated data flow. This dual-source approach ensures the platform's knowledge base is built on a foundation of both ground-truth technical data and essential human-generated context.

---

## **Phase A — Collection & Evidence Ingestion**

1. **Read-Only Discovery:** The Discovery & Ingestion Service uses specialized adapter connectors to perform read-only discovery against client source APIs (vCenter, AWS, Azure).

2. **Immutable Evidence Bundling:** All collected data (API responses, config files, architectural documents, spreadsheets) is packaged into an immutable, compressed EvidenceBundle in GCS. A SHA256 checksum is computed for data integrity.
3. **Event Publication:** The service publishes a DiscoveryEvent to a Pub/Sub topic, containing metadata and a GCS pointer to the evidence bundle and its checksum. This triggers the next phase.

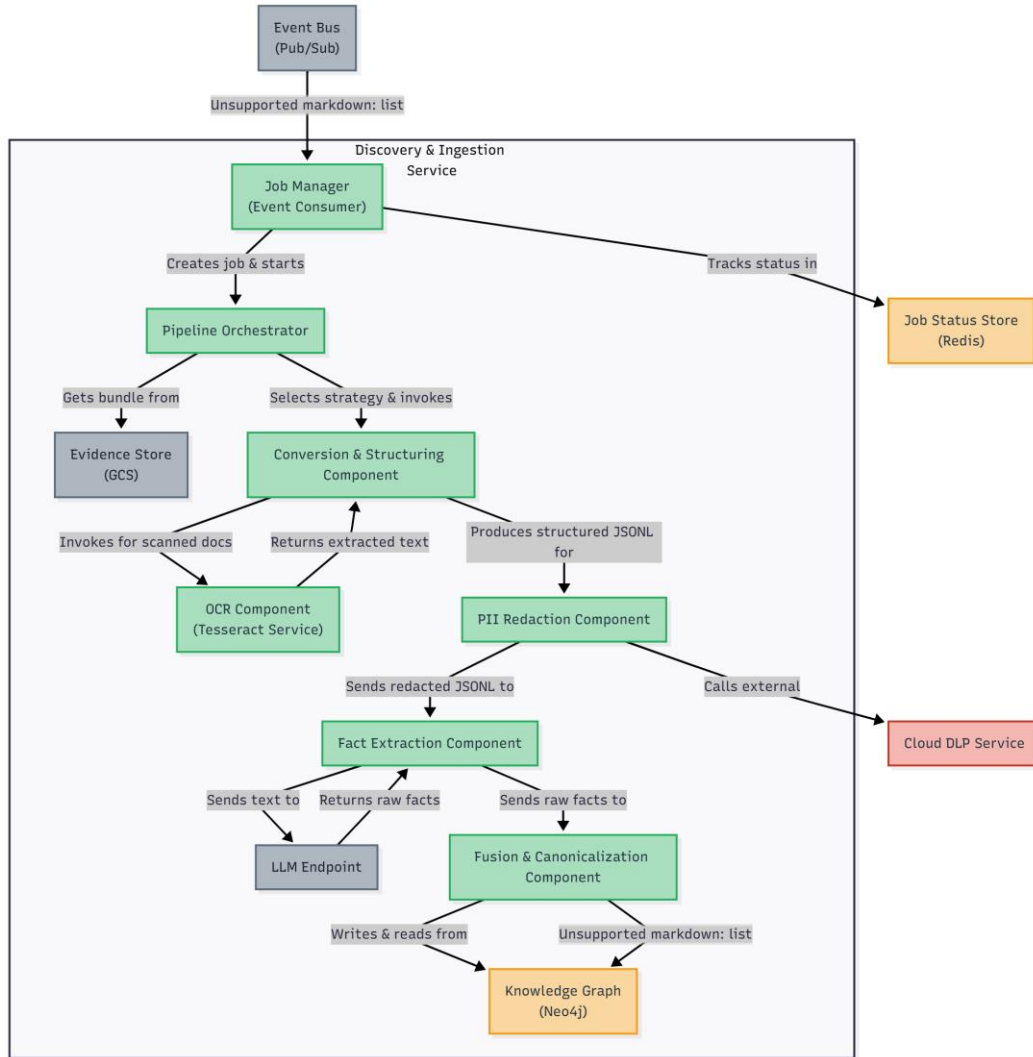
## Phase B — The Document Processing & Fusion Pipeline

This section provides a detailed architectural view of the **Discovery & Ingestion Service**, which executes the platform's sophisticated pipeline for transforming raw evidence into the canonical, **Stage 1 Foundational Knowledge Base**. This pipeline is designed for resilience, comprehensiveness, and scalability, ensuring that all subsequent agentic reasoning is built upon a high-quality, trusted foundation.

### Pipeline Principles

- **Resilience through Strategy Cascade:** No single parsing tool can handle all document types and corruptions. The pipeline uses a "cascade" of primary, secondary, and fallback strategies to maximize the chance of successful data extraction.
- **Comprehensiveness via Specialization:** The pipeline uses specialized tools for specific tasks—layout-aware partitioning for modern documents, OCR for scanned images, and streaming parsers for large files.
- **Quality through Fusion:** The pipeline's goal is not just to extract facts, but to create a canonical "single source of truth." The final Entity Fusion stage is a critical step that deduplicates and merges information from across the entire document corpus.
- **Traceability by Design:** Every piece of data in the final knowledge graph retains its provenance, linking it directly back to the source document, page, and even bounding box from which it was extracted.

### Pipeline Components (Internal Architecture)



*Data Processing*

The Discovery & Ingestion Service is composed of several internal micro-components that collaborate to execute the workflow:

### 1. Event Consumer & Job Manager:

- **Purpose:** The entry point for the service. It subscribes to the DiscoveryEvent topic on the Pub/Sub bus.
- **Function:** For each incoming event, it creates and manages a processing job, tracks its state in Redis, and orchestrates the subsequent pipeline steps. It is responsible for overall job status reporting.

### 2. Pipeline Orchestrator:

- **Purpose:** The "brain" of the processing pipeline.
- **Function:** It retrieves the EvidenceBundle from GCS. For each file, it inspects its metadata (file type, size) and intelligently invokes the appropriate components in the correct sequence (e.g., deciding whether to use streaming, or if OCR is likely required).

### 3. Conversion & Structuring Component:

- **Purpose:** To transform any raw document into a single, standardized structured format (JSONL).
- **Function:** This component contains the multi-strategy processing logic:
  - **Primary Strategy (Unstructured.io / MinerU):** Invoked for modern, text-based formats (PDF, DOCX, HTML). It performs high-resolution partitioning to identify logical elements like titles, tables, and lists.
  - **Streaming Parser:** Invoked by the Orchestrator for very large files (e.g., >50MB Excel or CSV files). It processes the file row-by-row or chunk-by-chunk to avoid high memory usage.
  - **Fallback Strategy (PyMuPDF):** If the primary strategy fails, this is used to perform a more basic text extraction.

### 4. OCR Component:

- **Purpose:** To extract text from images and scanned documents.
- **Function:** A containerized service running **Tesseract OCR**. The Conversion Component makes a service call to the OCR Component when it detects an image or a PDF with no selectable text. The extracted text is returned and integrated into the final JSONL structure.

### 5. Fact Extraction Component:

- **Purpose:** To identify entities and relationships within the structured text.
- **Function:** It takes the clean JSONL output from the Conversion Component and sends relevant passages (e.g., paragraphs, table rows) to a fine-tuned LLM. It uses carefully crafted prompts to extract structured **Raw Foundational Facts** (e.g., (Server: "srv-01")-[:HAS\_IP]->(IP: "10.1.2.3")).

### 6. Fusion & Canonicalization Component:

- **Purpose:** To deduplicate entities and create the canonical knowledge graph using a durable, AI-native workflow.
- **Function:** This component's logic is orchestrated as a **Temporal Workflow** to ensure reliability, retries, and observability, even for very large jobs.
  - **Workflow Trigger:** A new fusion workflow is started by the Pipeline Orchestrator.
  - **Sharding (Fan-Out):** The workflow begins by sharding the set of "unverified" entities into manageable batches.
  - **Candidate Retrieval (Activity):** For each batch, a Temporal Activity performs the hybrid search against Elasticsearch and Vertex AI to retrieve candidate matches.
  - **LLM Comparison (Parallel Activities):** The workflow then fans out, executing the pairwise LLM comparisons as parallel, independent Temporal Activities. This provides natural concurrency management and batching. If an LLM call fails or times out, Temporal will automatically retry that specific activity. If a Cloud Run instance fails, the activity will be re-run on another instance, with its state preserved.
  - **Deterministic Decision (Activity):** After the comparisons are complete, a final activity aggregates the judgments and applies the deterministic logic (merge, create new, or flag for human review), updating the Neo4j graph.

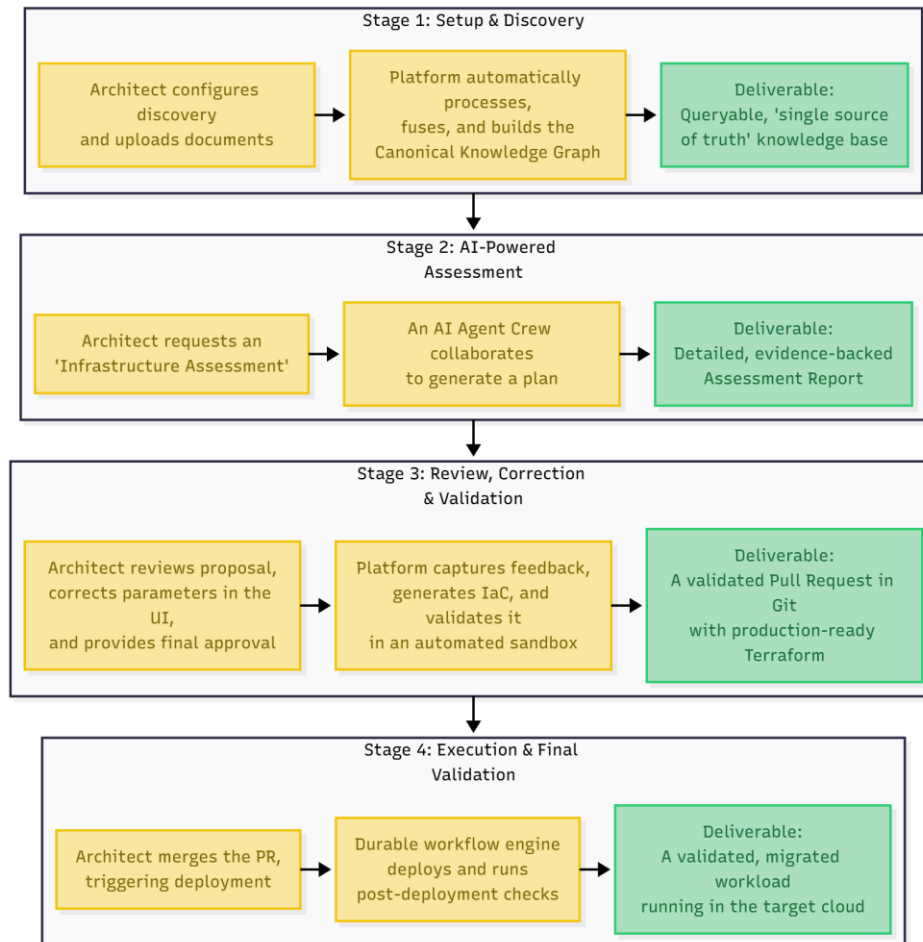
## 7. Fusion Decision Contract

The final "Deterministic Decision" step of the fusion workflow operates on a strict, auditable contract to ensure consistency and reliability.

- **Scoring & Aggregation:** The LLM's pairwise judgments are aggregated. The final `merge_score` for a candidate is a weighted sum of `LLM_confidence` and `textual_similarity_score` (from Elasticsearch).
- **Classification Thresholds:** A strict policy is applied to the final score:
  - `score > 0.95`: **AUTO-MERGE**. The action is executed and logged.
  - `0.75 < score <= 0.95`: **FLAG FOR HUMAN REVIEW**. The entities are linked in a "requires\_review" state and added to a UI queue.
  - `score <= 0.75`: **CREATE NEW CANONICAL**. A new canonical entity is created.
- **Canonical Node Versioning & Rollback:** The knowledge graph uses an immutable versioning model. A "merge" does not overwrite data; it creates a *new version* of the canonical node and updates a `preferred_version` pointer. This enables a simple, safe rollback mechanism. An incorrect merge can be reverted via an audited API call

(e.g., `POST /fusion/revert?canonical_id=...&to_version=...`), which simply reverts the preferred\_version to the last known good state.

## Detailed Process Flow



1. **Pipeline Invocation:** A DocumentProcessing workflow is triggered by the DiscoveryEvent.
2. **Multi-Strategy Conversion & Structuring:** The workflow routes each document in the bundle through a cascade of processing strategies to transform it into a structured JSONL format.
  - **Strategy 1 (Primary - High-Resolution Partitioning):** The document is first processed by **Unstructured.io** and **MinerU**. This strategy is optimized for modern file types (PDF, DOCX, etc.) and excels at preserving document



layout, accurately identifying elements (titles, tables, lists), and understanding reading order.

- **Strategy 2 (OCR Integration):** If the primary strategy detects a scanned PDF, an image, or a document with a low text-to-image ratio, it triggers the **Tesseract OCR** service to extract the text content. This text is then reintegrated into the structured output.
  - **Strategy 3 (Fallbacks):** If the primary strategies fail, a series of fallback parsers (PyMuPDF, Markdown) are attempted to ensure that, at a minimum, the raw text content is extracted.
  - **Streaming for Large Files:** If a document exceeds a configured size threshold (e.g., 50MB), specialized **streaming parsers** are used to process the file in chunks, ensuring the service never exceeds its memory limits.
3. **Fact Extraction:** The resulting structured JSONL is passed to a fact extraction module. This module uses a combination of LLM-powered analysis and fine-tuned models to identify and extract entities (e.g., servers, applications, databases) and their relationships from the text. The output is a set of raw **Foundational Facts**.
4. **Graph Ingestion:** The extracted facts are ingested into the Neo4j Graph Database. ... At this point, the graph may contain duplicate entities from different sources. To support strategic and organizational analysis, the graph ontology is extended to include nodes such as Team, Skill, BusinessObjective, and Stakeholder.
5. **Entity Fusion and Canonicalization:** A FusionOrchestrator process runs periodically or upon completion of a large ingestion. This is the critical step that creates a "single source of truth."
- **Clustering:** It identifies newly ingested entities and uses a multi-strategy matching algorithm (exact string, fuzzy matching, attribute similarity, and eventually semantic/vector similarity) to cluster duplicates.
  - **Canonicalization:** For each cluster, it either elects a canonical entity or creates a new one. It then merges the properties and relationships from all duplicates onto the single canonical node.
  - **Link Merging:** All original "raw" entities are then linked to the new canonical entity, preserving the audit trail while ensuring that all future queries can resolve to a single, authoritative node.

6. **Embedding and Indexing:** With the clean, structured, and deduplicated data now available, a final process chunks the content and computes vector embeddings, which are stored in **Vertex AI Matching Engine**. Keyword indexes are simultaneously built in **Elasticsearch**. The indexing pipeline ensures consistency. Upon a canonical entity update, an event is published that triggers a targeted re-indexing of that entity and its related documents in both Elasticsearch and the Vector DB, ensuring the search indexes remain in sync with the graph.

At the end of this phase, the **Stage 1 Knowledge Base** is complete. It is a rich, interconnected, and canonical representation of the client's enterprise, ready to be used by the intelligent agent crews.

## 8. Step-by-Step Workflow: Stage 2 Insights Synthesis

With the high-fidelity Stage 1 Foundational Knowledge in place, the platform moves to its primary value-creation phase. This is performed by the **Reasoning & Proposal Service**, which operates in two distinct modes:

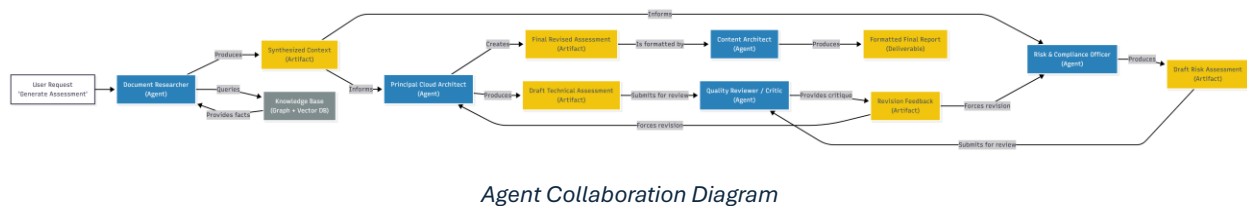
- **Evidence-Based Synthesis:** The default mode, where agent crews perform bottom-up reasoning based entirely on the foundational facts discovered in Stage 1. All recommendations are directly traceable to source evidence.
- **Assumption-Driven Modeling:** A top-down mode for strategic planning, where architects provide high-level goals, constraints, and assumptions as inputs. This allows agents to generate strategic artifacts like business cases or future-state visions that are not strictly tied to the current-state evidence.

### Phase C — Structured Workflow Execution with CrewAI

This phase is triggered when a user requests a deterministic, high-quality deliverable, such as a "Comprehensive Infrastructure Assessment."

1. **API Invocation & Crew Assembly:** The user makes a request to the FastAPI endpoint (e.g., POST /crews/infrastructure-assessment/execute). The Reasoning & Proposal Service receives this, validates it, and consults its internal registry to assemble the appropriate **CrewAI** crew. For this task, it instantiates the Infrastructure Assessment Crew:
  - Document Researcher
  - Principal Cloud Architect
  - Risk & Compliance Officer

- Quality Reviewer (Critic)
  - Content Architect
2. **Tool Binding & Context Injection:** Each agent in the crew is dynamically equipped with a set of tools. These tools are provided by **Semantic Kernel**, which acts as the universal adapter. The tools include:
- RAGQueryTool: To perform semantic and keyword searches against the Vector DB and Elasticsearch.
  - GraphQueryTool: To execute Cypher queries against the Neo4j Knowledge Graph.
  - ComplianceFrameworkTool: A specialized tool that contains the rules for various compliance regimes.
  - **MCP-based Tools:** If enabled, the service dynamically discovers and binds tools from external Model Context Protocol servers (e.g., a live AWS Pricing tool).



Agent Collaboration Diagram

## Secure Tool Binding & MCP Enforcement

Dynamically binding tools to agents is powerful but presents a security risk. The platform enforces least-privilege for tools using the following model:

1. **Capability Manifest:** Each tool exposed via the MCP Gateway is defined by a **Capability Manifest**, a JSON document that declares its functions, required parameters, and, most importantly, its **access scope** (e.g., aws:pricing:read, aws:s3:list).
2. **Crew-Scoped Capability Tokens:** When a CrewAI workflow is initiated, the Reasoning Service does not grant agents direct access to the MCP Gateway. Instead, it determines the total set of scopes required by the crew's agents and mints a short-lived (e.g., 1-hour TTL) **Capability Token** (a JWT). This token contains the list of approved scopes for that specific job.
3. **Authenticated & Audited Tool Calls:**

- When an agent calls a tool, Semantic Kernel passes the request to the MCP Gateway, including the Capability Token.
  - The MCP Gateway validates the token's signature and TTL.
  - It then checks if the requested tool's scope (from its manifest) is present in the token's claims.
  - If authorized, the gateway executes the call to the external MCP server.
  - Crucially, every single tool request and its authorization decision is logged to an **immutable audit log** (e.g., a write-once GCS bucket or BigQuery table), including the agent name, job ID, requested tool, and the outcome.
3. **Sequential Task Execution:** The service kicks off the crew's pre-defined sequential workflow (Process.sequential).
- **Task 1 (Researcher):** The Document Researcher is given the initial goal. It uses the RAGQueryTool and GraphQueryTool to gather all foundational facts about the target workloads, synthesizing them into a structured context document.
  - **Task 2 (Architect & Officer):** The Principal Cloud Architect and Risk & Compliance Officer work in parallel. The Architect analyzes the context to identify modernization opportunities and drafts technical recommendations. The Officer analyzes it against compliance rules and drafts a risk assessment.
  - **Task 3 (Reviewer):** The Quality Reviewer receives the drafts from both experts. This is a critical step. It cross-references every claim against the context provided by the Researcher, checks for contradictions, and validates the logical flow. It then provides structured feedback and *rejects* the drafts, sending them back for revision.
  - **Task 4 (Revision):** The Architect and Officer revise their drafts based on the Reviewer's precise feedback.
  - **Task 5 (Final Synthesis):** The revised drafts are passed to the Content Architect. This agent's job is not to reason about the content, but to structure it. It combines the technical and risk assessments into a single, professional, human-readable document, adding an executive summary, table of contents, and proper formatting.

4. **Result Persistence & Notification:** The final document artifact is stored in GCS. The Reasoning & Proposal Service then persists the final Proposal object (with links to the evidence and the generated document) in the Cloud SQL database and publishes a ProposalReady event to the Pub/Sub bus to notify the UI.

## Phase D — Interactive Conversational Assistance with AutoGen

This phase is used for ad-hoc queries, "what-if" simulations, and interactive problem-solving via the platform's chat interface.

1. **WebSocket Connection & Session Start:** The user connects to the chat UI, which establishes a **WebSocket** connection to the Reasoning & Proposal Service. A new conversation session is created and persisted in the PostgreSQL database.
2. **Query & Agent Selection:** The user submits a query (e.g., "What are the cost implications of migrating the 'CRM-DB' server to GCP using Cloud SQL vs. running it on a C3 instance?"). The service analyzes the query's intent and selects a team of appropriate **AutoGen** conversational agents:
  - user\_proxy: Represents the human user in the chat.
  - Principal Cloud Architect
  - Cost Optimizer
  - Data Expert
3. **Collaborative Conversation:** An AutoGen GroupChatManager initiates a multi-turn conversation between the selected agents to resolve the query.
  - The Architect might start by using the GraphQLQueryTool to look up the specs of 'CRM-DB'.
  - The Data Expert might add context about the database type and potential Cloud SQL compatibility issues.
  - The Cost Optimizer would then take this information, use a live pricing tool (via MCP) to fetch current costs for both Cloud SQL and C3 instances, and perform a comparison.
  - Throughout this process, the conversation is streamed in real-time to the user's UI via the WebSocket.

4. **Conversation Persistence:** Each message from each agent is persisted in the `conversation_messages` table in PostgreSQL, linked to the session ID. This ensures that the context is maintained for follow-up questions.
5. **Human-in-the-Loop:** If the agents need clarification, they can direct a question to the `user_proxy`, pausing the conversation until the human provides input through the chat interface.

## Phase E — Durable Execution with Temporal

This phase begins after a human architect has reviewed and approved a Proposal and merged the resulting IaC pull request. It is managed by the **IaC & Validation Service**.

1. **CI/CD Trigger:** A merge to the main branch of the IaC repository triggers a webhook.
2. **Temporal Workflow Invocation:** The webhook is received by the IaC & Validation Service, which translates it into a `StartWorkflow` call to the **Temporal** service. A new, durable workflow instance for `Execute-Migration-<workload-id>` is created.
3. **Stateful, Long-Running Execution:** The Temporal workflow now orchestrates the entire deployment and validation process, which could take hours. Each step is a durable "Activity."
  - **Activity 1: RunTerraformApply:** A Temporal worker securely retrieves credentials, executes terraform apply, and checkpoints the state. If the worker crashes, Temporal will automatically restart the activity on another worker.
  - **Activity 2: RunPostflightChecks:** After the infrastructure is provisioned, this activity runs a series of API calls and scripts to verify its health and configuration.
  - **Activity 3: WaitForDataSync (if applicable):** The workflow can have durable timers, waiting for a separate data migration process to complete.
  - **Activity 4: RunValidationTests:** Executes the final validation smoke tests.
4. **Compensation Logic (Rollback):** If any activity fails after a series of retries, the workflow can execute compensation logic, such as automatically triggering a terraform destroy to ensure the environment is left in a clean state.
5. **Completion & Notification:** Upon successful completion, the workflow updates the status in the Cloud SQL database and publishes a `MigrationComplete` event.

The entire execution history, including logs and timings for every activity, is available for audit in the Temporal UI.

## Phase F — Continuous Learning & Improvement

This final phase is orchestrated by the **Improvement Service** and closes the loop, ensuring the platform gets smarter over time.

1. **Capture All Feedback Signals:** The service subscribes to events from across the platform: `ProposalRejected`, `PRModified` (with a git diff), and `SimulationVsRealityDelta` from post-deployment monitoring.
2. **Pattern Analysis & RDL Updates:** The service's Post-Processing Agent periodically analyzes these signals to find patterns. If it finds a consistent pattern (e.g., architects always increase memory for Java apps), it formulates a suggested change to the RDL and raises a pull request for human review.
3. **MLOps for Model Retraining:** The feedback signals create a valuable human-labeled dataset. This data is fed into a **Vertex AI Pipeline** to automatically fine-tune the RAG reranker models or recalibrate the confidence rubric weights, ensuring the platform's AI components adapt and improve.

### Phase F - 1.b: The Human Correction & Feedback Workflow

The platform's ability to learn is critically dependent on the quality of the feedback signals it receives from human architects. The most valuable signals are direct corrections. The platform will facilitate this through a dedicated UI workflow and a sophisticated backend process.

- **The "Review & Correct" Interface:** The Management Portal UI will not be limited to a simple "Approve/Reject" button. When reviewing a Proposal, the architect will be presented with the key parameters and decisions in an editable form.
- **Dynamic, Template-Driven UI:** The UI for corrections is designed to be scalable and not rely on hardcoded fields. The editable parameters for a given proposal are dynamically rendered based on metadata defined within the **RDL Decision Template** itself.
  - **Mechanism:** The RDL template for "VM Rightsizing" will contain a `ui_parameters` section that defines which fields are user-editable (e.g., `machine_type`, `disk_size`) and what their validation rules are (e.g., string, integer).
  - **Example:** Instead of a simple dropdown, the UI for `machine_type` would be a searchable text field with an autocomplete function. This function makes a

live API call to the target cloud provider (scoped by the project's credentials) to fetch a list of currently available and valid machine types, preventing the user from entering an invalid option. This makes the UI dynamic, scalable, and always up-to-date with the cloud provider's offerings.

- **Capturing Structured Feedback:** When the architect submits this change, the frontend doesn't just send the final state. It sends a structured event to the backend, `ProposalCorrected`, which contains:
  - The proposalId.
  - The original proposed value (n2-standard-4).
  - The architect's corrected value (n2-highmem-4).
  - The field that was changed (`machine_type`).
  - Rich context about the workload (e.g., tags like `app:java`, `env:prod`).

**High-Quality Signal for the Improvement Service:** This structured event is a far more powerful learning signal than a simple git diff. The Improvement Service consumes these events and can immediately identify patterns. If it sees multiple corrections from standard to highmem for workloads tagged with `app:java`, it has a high-confidence, statistically valid reason to propose an update to the underlying decision template in the RDL. This creates a tight, rapid, and data-driven loop for refining the platform's automated reasoning. These `ProposalCorrected` events are persisted indefinitely in a dedicated BigQuery table, forming a high-quality, human-labeled training dataset for future MLOps initiatives.

---

## 9. Final Architectural Considerations

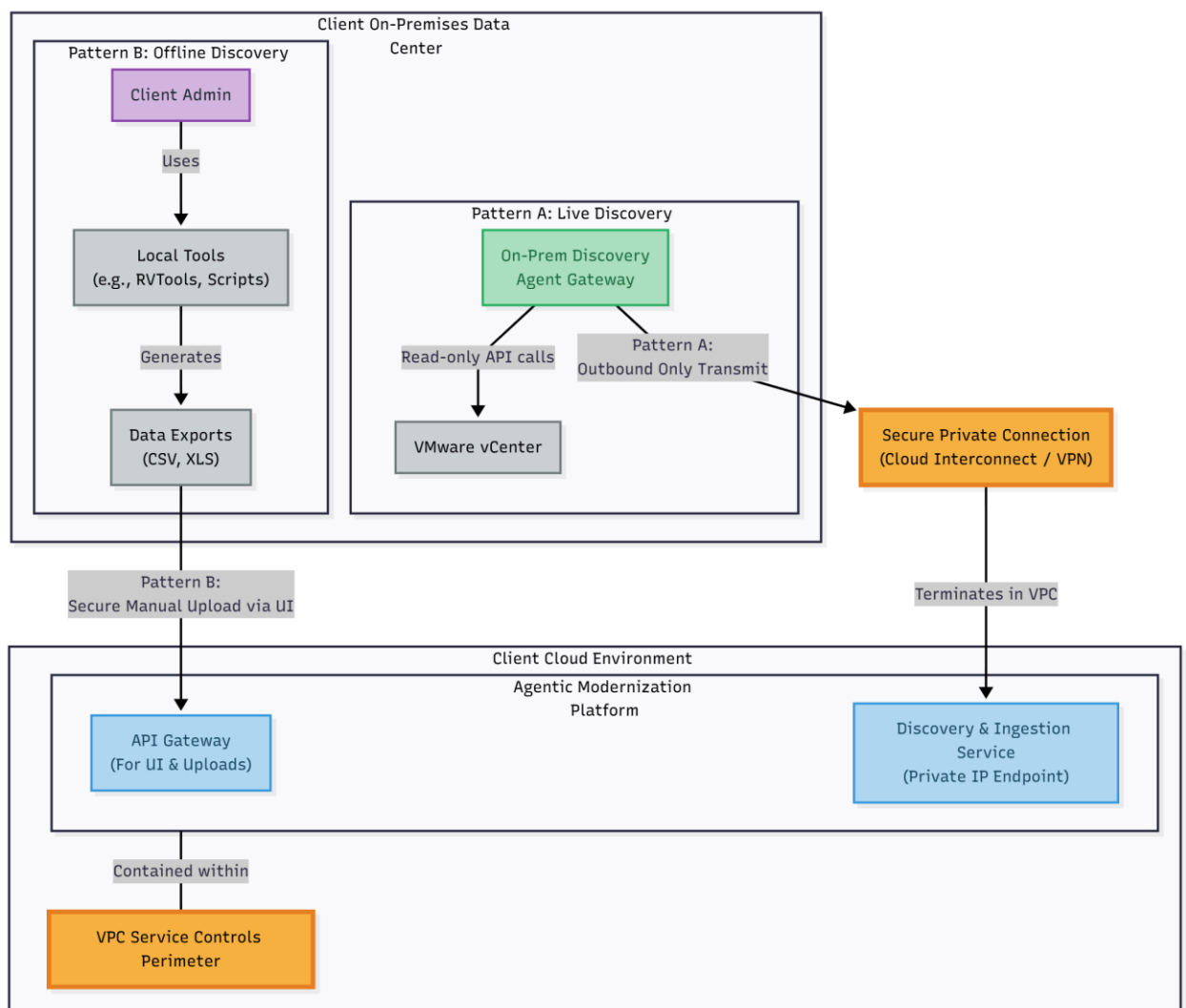
### 9.1. Infrastructure & Network Security

- **Perimeter Control with VPC Service Controls:** The entire platform, including all its services and data stores, will be deployed within a strict VPC-SC perimeter. This acts as a virtual data firewall, preventing data exfiltration and ensuring that services can only communicate with authorized Google Cloud APIs, effectively blocking unintended egress.



- **Zero-Trust Internal Networking with mTLS:** All communication *between* the platform's internal services (e.g., Reasoning Service to Discovery Service) will be encrypted using mutual TLS (mTLS). This will be enforced by deploying the platform on a service mesh like **Anthos Service Mesh**, which automates certificate issuance, rotation, and traffic policies, guaranteeing that no unauthenticated or unencrypted traffic is allowed within the cluster.
- **Private Egress for External APIs:** All outbound calls to external LLM endpoints are routed through a secure egress path using **Private Google Access** and a NAT Gateway. The NAT Gateway's firewall rules will have a strict allowlist, permitting traffic only to the pre-approved IP address ranges of our sanctioned LLM providers (e.g., Vertex AI, Azure OpenAI).

*This multi-layered network security strategy aligns with the core principles of the NIST Zero Trust Architecture (NIST SP 800-207).*



### 9.1.b. Hybrid & On-Premises Deployment Topology

- The definitive deployment model is a hybrid architecture where the platform's **control plane runs in a client-owned cloud project**, and for on-premises discovery, a **lightweight, stateless, outbound-only gateway is deployed in the client's data center**. In scenarios where even this is not permitted, the platform can operate in a "document-only" ingestion mode where data exports are manually uploaded.
- When the platform is deployed to support discovery of a client's on-premises environment (e.g., VMware), the following hybrid network architecture is required to guarantee data residency and enforce strict egress controls.
- **Deployment Model:** The core platform (Control Plane, Reasoning Services, etc.) runs in a dedicated, client-owned cloud project (the "Cloud Plane"). A lightweight, stateless **Discovery Agent Gateway** container is deployed on-premises within the client's data center (the "On-Prem Plane").
- **Network Connectivity:** All communication between the On-Prem and Cloud planes is **initiated from the on-premises gateway to the cloud**. This is achieved via a secure, private connection, typically **Cloud Interconnect** or a **site-to-site VPN**, terminating in the client's VPC. No inbound connections from the cloud to the on-prem network are ever required or permitted.
- **Data Flow & Residency:**
  - The on-prem Discovery Agent Gateway receives discovery jobs from the cloud-based Discovery & Ingestion Service via this secure channel.
  - The gateway then performs the read-only discovery against the local vCenter APIs.
  - The raw data is packaged into an EvidenceBundle *on-premises*. This bundle is **encrypted at rest** on the local gateway's volume using **AES-256** before transmission.
  - This bundle is then securely transmitted over the private interconnect to the GCS bucket in the Cloud Plane for processing.
- **Egress Control & Offline Mode:** The on-prem gateway is deployed in a highly restricted network segment. Its only permitted egress route is to the specific private IP address of the Discovery & Ingestion Service endpoint in the cloud VPC. All other outbound internet access is blocked. The gateway is designed to be resilient; if the connection to the cloud is lost, it will securely queue collected data locally (to an

encrypted volume) and resume transmission once the connection is restored. No telemetry or usage data leaves the client premises unless explicitly defined and authorized in a data sharing agreement.

---

## 9.2. Identity, Access, and Supply Chain Security

- **Granular, Role-Based IAM:** The principle of least privilege is enforced everywhere. Each microservice and CI/CD job runs with its own dedicated IAM service account with a single, specific purpose. For instance, the IaCAgent-SA has permissions only to create a pull request, while the CICD-Apply-SA is the only identity with terraform apply permissions, and those permissions are granted just-in-time for the deployment job's execution.
- **Supply Chain Security with SLSA & Attestation:** We will adopt the SLSA framework to provide verifiable guarantees about the integrity of our software. This is critical for building client trust when our IP is deployed as a "black box."
  - **Verified Builds:** The build process is hardened and runs in an ephemeral, isolated environment.
  - **Provenance Generation:** The pipeline generates authenticated, non-forgable metadata (provenance) that details exactly how an artifact was built, including the source commit hash and the builder identity.
  - **Binary Authorization:** We will use Google Cloud's Binary Authorization service, which enforces a policy that only allows container images with verified SLSA provenance to be deployed into our GKE environment.
  - **Signed Artifacts & Attestations:** Every production artifact (container image, RDL artifact) will be cryptographically signed using Sigstore. The SLSA provenance document itself will also be signed. These signatures and the SBOM (Software Bill of Materials) for each artifact will be published to a client-accessible GCS bucket.
  - **Client-Facing Verification API & Runbook:** We will provide a simple verification API or a documented runbook that allows the client's security team to independently verify the integrity of any running component. This process involves:
    - Querying the running container's digest.
    - Fetching the corresponding signature, provenance, and SBOM from the attestation bucket.

- Using public keys and open-source tools (like cosign) to cryptographically verify that the running artifact is exactly what we built and that its contents match the provided SBOM.
- **Understanding SLSA Attestations:** In plain language, a SLSA attestation is a cryptographically signed, tamper-proof "birth certificate" for a piece of software. It is a structured metadata file that proves exactly how, when, and from what source code an artifact was built. The digital signature ensures this "birth certificate" is authentic, and Binary Authorization acts as a verifier that only allows software with a valid, trusted certificate to run in the client's environment.
- **Security, Governance, and Privacy by Design** (Detailing VPC-SC, mTLS, Granular IAM, Supply Chain Security).
- **Platform CI/CD and Operational Model** (Detailing the polyrepo structure and Cloud Build pipelines).
- **Performance, Scaling, and Cost Considerations.**
- **Phased Rollout & Immediate Next Steps** (Detailing the incremental spike plan).

## 9.3 Prompt Lifecycle Management

In an AI-native system, prompt templates are a form of high-leverage source code that directly governs the behavior and quality of the AI agents. As such, they will be managed with the same rigor as the platform's application code.

- **Centralized Prompt Repository:** All system and agent prompts will be stored in a dedicated Git repository named prompt-library. Prompts will be stored in a human-readable format (e.g., .yaml or .md files) that separates the instruction text from metadata (e.g., version: 1.2, model: gemini-1.5-pro, author: arch-team).
- **Versioning and Change Control:** Any change to a prompt requires a pull request, which must be peer-reviewed. This ensures that changes are deliberate, documented, and understood by the team. The repository will use semantic versioning to track prompt evolution.
- **CI/CD for Prompts:** The prompt-library repository will have its own dedicated CI/CD pipeline in Cloud Build.
  - **Linting & Validation:** The pipeline will automatically lint the prompt files for syntax and metadata correctness.
  - **Evaluation Against Golden Dataset:** For critical prompts (e.g., the one that generates the final assessment), the pipeline will automatically run the new prompt version against a "golden dataset" of test inputs. It will compare the

AI's output to a set of pre-approved "golden responses" and calculate quality metrics (e.g., correctness, verbosity, format adherence). A significant regression in these metrics will block the PR from being merged.

- **Dynamic Loading:** The Reasoning & Proposal Service will be configured to load prompts dynamically from this versioned library, rather than having them hardcoded. This allows for the rapid iteration and improvement of prompts without requiring a full redeployment of the agent services.

## 9.4 Scalability of the Knowledge Fusion Process

The LLM-powered fusion process is designed to be both semantically intelligent and scalable.

- **Default Strategy (Cloud Run):** For the vast majority of enterprise estates (up to millions of entities), the default strategy of using Cloud Run for parallel LLM comparisons is the most efficient. Sharding and concurrency are managed by the Temporal workflow, which fans out activities. Throttling and backpressure are handled by configuring the max concurrency of the Temporal worker pool to stay within LLM API rate limits.
- **Scaling Playbook (Fallback to Big Data):** For extreme-scale engagements where the number of candidate pairs generated by the filtering step exceeds a predefined threshold (e.g., >10 million pairs), the cost and latency of individual LLM calls become prohibitive. In these specific cases, the platform will switch to a **fallback big data strategy**:
  - The candidate pairs are exported to GCS.
  - A **Dataprox Serverless (Spark) job** is triggered.
  - This job uses traditional, deterministic algorithms (e.g., Jaro-Winkler distance, attribute matching) to perform a brute-force comparison on the massive dataset, producing a list of high-confidence matches.
  - This list is then ingested back into the graph.
- **Cost Control:** The estimated LLM cost per shard is calculated before execution. If the total estimated cost for a fusion job exceeds the project's budget, the job is paused and requires manual operator approval to proceed.

## 9.5. Key Performance Indicators (KPIs) and Service Level Objectives (SLOs)

While the platform tracks detailed technical metrics, its success is ultimately measured by its impact on the business process. The primary operational dashboard will focus on a handful of key, business-relevant KPIs and the SLOs that support them.

- **Primary Business KPI: Migration Velocity (Lead Time to PR)**
  - **Definition:** The average time elapsed from the creation of an assessment project to the generation of an architect-approved pull request for the target workloads.
  - **Business Goal:** To reduce the current manual assessment and design time by a target of 50% within the first year.
- **Primary Quality KPI: Proposal Acceptance Rate**
  - **Definition:** The percentage of AI-generated proposals that are approved by an architect (including those approved with minor modifications).
  - **Business Goal:** To achieve a >80% acceptance rate, indicating that the platform's recommendations are consistently high-quality and trustworthy.
- **Key Platform SLOs:** These are the technical objectives that the platform must meet to support the KPIs.
  - **SLO 1: Data Ingestion Freshness:** 99% of discovery runs from configured adapters must be successfully ingested and reflected in the knowledge graph within 24 hours of execution.
  - **SLO 2: Proposal Generation Latency:** The P95 latency for a DecideAgent crew to generate a proposal for a medium-sized application (e.g., <10 workloads) must be less than 5 minutes.
  - **SLO 3: Platform Availability:** The core API endpoints for proposal review and interactive chat will have a 99.9% availability SLO.

### 9.5.b. Performance Envelope & Cost Targets

To support the KPIs and SLOs, and to provide a baseline for financial governance, the platform is designed against the following initial performance and cost targets. These will be refined based on pilot engagement data.

| <i>Metric</i>                            | <i>Target</i>                 | <i>Notes</i>                                                                                              |
|------------------------------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b><i>Avg. Tokens per Document</i></b>   | < 20,000 tokens               | Assumes a mix of fact extraction and summarization for a standard 10-20 page document.                    |
| <b><i>Avg. Tokens per Proposal</i></b>   | < 150,000 tokens              | Includes the full CrewAI workflow (research, drafting, critique, revision) for a medium-sized assessment. |
| <b><i>Max Fusion Cost per Entity</i></b> | < 500 input tokens            | Target for the candidate retrieval and LLM comparison workflow for a single new entity.                   |
| <b><i>Concurrency Limit</i></b>          | 50 concurrent agent workflows | Initial autoscaling target for the Reasoning Service to balance throughput and cost.                      |
| <b><i>P99 RAG Latency</i></b>            | < 2,500 ms                    | The end-to-end latency for the full hybrid retrieval and reranking pipeline.                              |

The CI/CD pipeline for the Reasoning Service will include an integration test that runs a benchmark assessment and fails the build if the token consumption for that benchmark exceeds a predefined threshold, preventing cost regressions.

## 9.6 AI Safety & Security Layer

Beyond traditional infrastructure security, the platform implements a specific layer of safety controls to mitigate the unique risks associated with generative AI and agentic systems.

- **1. Prompt Injection Defense:** All user-provided input, especially in the interactive chat interface, is treated as untrusted.
  - **Mechanism:** We will implement a multi-layered defense. First, input is sanitized to remove known malicious payloads. Second, prompts are constructed using robust templating with clear delimiters between instructions, context, and user input. Third, a GuardianAgent (a specialized LLM call) may be used to inspect the final assembled prompt before it is sent to the primary LLM, specifically looking for attempts to subvert the original instructions.
- **2. Malicious IaC Prevention:** The primary risk is an agent being tricked into generating malicious or destructive IaC (e.g., terraform destroy on a production environment, or opening a firewall to 0.0.0.0/0).
  - **Mechanism:** This is mitigated by a defense-in-depth approach:

- **RDL Guardrails:** The Reference Decision Library will contain hardcoded Rego policies that explicitly forbid dangerous configurations.
- **OPA Validation:** The IaCAgent is required to run the generated Terraform plan through the **Open Policy Agent (OPA)** before creating a pull request. An OPA policy will explicitly fail any plan that contains destructive actions on critical resources or violates core security rules. This is a deterministic, non-AI-based safety check.
- **Human-in-the-Loop:** The final and most important backstop is that no IaC is ever executed automatically. It is always presented as a pull request for mandatory review by a human architect.
- **3. Containing Agent Actions (Tool Safety):** Agents are given powerful tools (e.g., the ability to query cloud APIs).
  - **Mechanism:** Tools provided to agents will be strictly scoped with least-privilege principles. No agent will ever be given a tool capable of making write/delete/update calls to a live client environment. All generative actions terminate at the creation of a pull request. The only service with write permissions is the CI/CD pipeline, which is triggered by a human-approved merge.

### 9.6.b. Tenant Isolation and Prompt Sandboxing

The platform is designed for a client-hosted, single-tenant deployment model, which provides a strong baseline of isolation. However, within the platform, we enforce further layers of logical separation to guarantee that no data can leak between engagement projects and to sandbox AI interactions.

- **Per-Project Data Encryption:** While CMEK is used for the entire project, all data at rest in the **Vector DB** will be stored in separate, per-project collections or namespaces. Furthermore, where supported, data will be encrypted with a per-project key to provide an additional layer of cryptographic isolation.
- **Strict Data Scoping:** All database queries (to SQL, Neo4j, or the Vector DB) made by an agent are mandatorily scoped by the projectId derived from the user's session or the job's context. This is enforced at the data access layer. An agent working on Project A can never accidentally query data from Project B.



- **laC Sandbox Validation:** As detailed in the CI/CD workflow, all generated laC is validated in an **ephemeral, isolated "sandbox" project**. This provides a critical security boundary, proving that the code is safe and functional before it is ever proposed for execution against a real environment.

### 9.6.c. PII Handling, Deletion, and Secure Viewing

Even in a single-tenant environment, PII is handled with specific controls to meet client compliance requirements and to secure the AI reasoning process. The primary goal is to **prevent sensitive PII from ever being included in prompts sent to external LLM endpoints**.

| <i><b>Data Location</b></i>                | <b>PII Handling Strategy</b>                                                                                                   | <b>Justification</b>                                                                                                                                                                    |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i><b>Evidence Store (GCS)</b></i>         | Encrypted as-is (raw)                                                                                                          | The immutable, access-controlled source of truth.                                                                                                                                       |
| <i><b>Vector DB &amp; Search Index</b></i> | <b>Redacted.</b> PII is discovered and replaced with placeholders (e.g., [PERSON_NAME]) during the document structuring phase. | <b>This is critical control.</b> It guarantees that no PII is ever included in the RAG context and subsequently sent to the LLM for processing, satisfying strict client data policies. |

- **Secure Viewing (No Rehydration):** The platform **does not perform rehydration**. If an architect needs to view the original, unredacted PII, the process is simple and secure:
  1. The UI provides a "View Source" link next to any redacted content or evidence citation.
  2. Clicking this link (an audited event) directs the architect to the original, raw document in the **Evidence Store (GCS)**.
  3. The architect can then view the original document directly, using their own authorized access. This ensures that the platform's agentic and data processing layers never handle unredacted PII after the initial ingestion scan.
- **Deletion:** Upon client request for deletion, a documented and audited workflow is triggered. This workflow performs a cryptographic shredding of the data in all data stores associated with the project.

## 9.7. Model Serving and Lifecycle Management

The platform's performance and cost are directly tied to the LLMs it consumes. These models are treated as critical, versioned dependencies with a formal governance lifecycle.

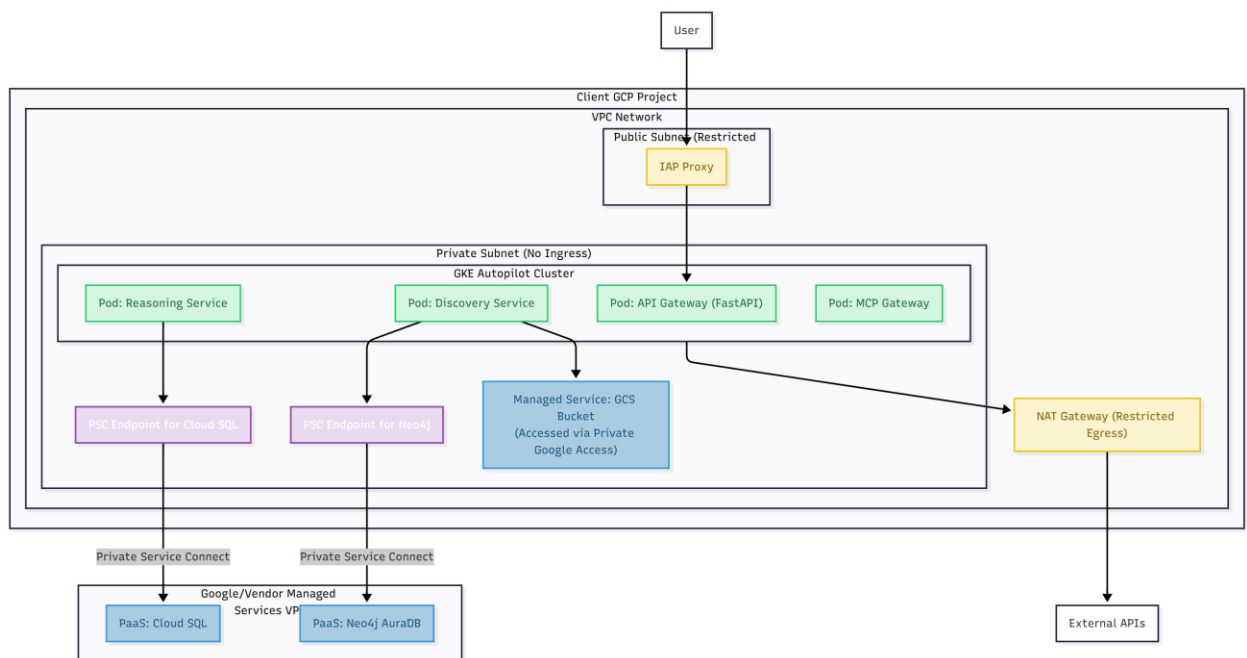
- **Inference Abstraction Layer:** All services will interact with LLMs through a dedicated internal **Inference Abstraction Layer**. This library provides a single, consistent interface for making model calls, and it contains the logic to route requests to the appropriate provider (e.g., Vertex AI, Azure OpenAI) based on configuration. This ensures multi-cloud portability.
- **Model Lifecycle Workflow:**
  1. **Sourcing & Registration:** A new model version (e.g., gemini-2.5-pro-002) is first registered in the **Vertex AI Model Registry**.
  2. **Evaluation:** The model is automatically evaluated against the "golden dataset" (from the prompt CI/CD pipeline) to measure its performance on key tasks. A report comparing its accuracy, cost, and latency against the current production model is generated.
  3. **Approval:** The model and its evaluation report are presented to the designated **RDL Maintainers** for a formal approval decision.
  4. **Deployment:** Once approved, the model's version is updated in the platform's configuration, and the Inference Abstraction Layer begins routing requests to the new version.
- **Fallback Strategy:** If the primary production model (e.g., Gemini 1.5 Pro) becomes unavailable or experiences a significant latency spike, the Inference Abstraction Layer will automatically:
  1. Trigger a high-priority alert to the Platform Operator.
  2. **Fallback** to a pre-configured secondary model (e.g., a version in a different region, or a smaller model like Gemini Flash).
  3. Log the fallback event for analysis.

## 10. Platform CI/CD and Operational Model

### 10.1. The Polyrepo Structure

To promote independent development, clear ownership, and aligned CI/CD, the platform will use a multi-repository ("polyrepo") structure in source control:

- platform-infra/: Contains the master Terraform code for provisioning the core platform infrastructure (GCP projects, networking, databases, IAM roles).
- service-<name>/: Each major service boundary (e.g., service-discovery/, service-reasoning/) will have its own repository. This contains the service's application code, Dockerfile, and unit/integration tests.
- rdl/: The dedicated, governed repository for the Reference Decision Library.
- prompt-library/: The dedicated, governed repository for all AI prompt templates.



Deployment Blueprint Diagram

### 10.2. Automated CI/CD Pipelines

All build and deployment processes are defined as code using **Cloud Build** (cloudbuild.yaml files within each repository).

- **Infrastructure Pipeline (platform-infra):** A push to the main branch triggers a terraform plan. A manual approval step is required in the Cloud Build UI before a terraform apply is executed to prevent accidental infrastructure changes.
- **Service Deployment Pipeline (service-\*):** A pull request triggers a pipeline that runs unit tests, performs static code analysis, and runs security scans. A merge to the main branch continues the pipeline:
  1. Builds the container image.
  2. Generates SLSA provenance.
  3. Pushes the image and provenance to **Artifact Registry**.
  4. Performs a controlled, automated rollout to Cloud Run or GKE using a **blue/green deployment strategy**. The pipeline automatically monitors for errors after the new version receives a small percentage of traffic and will trigger an automatic rollback if key SLOs are violated.

### 10.2.b. Federated CI/CD Workflow for Client-Managed Repositories

For enterprise clients who require that all Infrastructure as Code resides in their own source control and is deployed by their own CI/CD pipelines, the platform supports a "gated" or "federated" workflow. In this mode, the platform's role shifts from "executor" to "author and observer."

1. **PR Generation:** The Generation & Execution Service uses a securely scoped, short-lived credential to create a pull request directly in the **client's designated Git repository**.
2. **Client Pipeline Trigger:** This PR automatically triggers the **client's own CI/CD pipeline** (e.g., Jenkins, GitHub Actions). This pipeline is owned and managed entirely by the client.
3. **Client-Side Validation:** The client's pipeline executes all their mandatory security scans, compliance checks, and a terraform plan.
4. **Status Monitoring via Webhooks:** The Generation & Execution Service subscribes to status webhooks from the client's Git provider. As the client's pipeline completes its checks, the status is updated in the platform's UI, providing the architect with end-to-end visibility.
5. **Human Approval & Merge:** The PR follows the client's standard internal approval process.

6. **Client-Side Apply:** The final terraform apply is executed by the client's pipeline after the merge. The platform logs the completion state but does not participate in the execution. This model ensures that the platform can provide its value (automated IaC generation) while fully respecting the client's security and change management boundaries.

## 10.3. Operational Personas

For day-to-day operations, specific roles are responsible for managing different aspects of the platform.

| <i>Persona</i>              | Key Responsibilities                                                                                         | Primary Tools / Interfaces                   |
|-----------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <b>Platform Operator</b>    | Manages the MCP Gateway, certifies tools, monitors platform health and costs, and handles incident response. | <b>Ops Console</b> , Cloud Monitoring        |
| <b>Engagement Architect</b> | The primary user. Runs discovery, reviews proposals, uses interactive chat, provides feedback via the UI.    | <b>AI Management &amp; Collaboration Hub</b> |
| <b>Security Officer</b>     | Reviews and verifies SLSA attestations, audits tool certifications, and reviews high-risk RDL changes.       | <b>Attestation Bucket</b> , Audit Logs       |
| <b>DevOps Lead</b>          | Manages the IaC for the platform itself and oversees the CI/CD pipelines for all services.                   | <b>Cloud Build</b> , Terraform               |

---

## 11. Performance, Scaling, and Cost Considerations

### 11.1. Performance & Scaling Strategy

- **Stateless, Serverless Compute:** The primary compute platform is **Cloud Run**, a serverless container solution. This allows each service to scale independently from zero to thousands of instances based on incoming traffic (API calls or Pub/Sub messages), providing both extreme scalability and cost-efficiency.
- **Independent Data Tier Scaling:** The data layer is composed of managed services that can be scaled independently to meet specific performance demands. For example, we can increase the vCPUs and memory for the Cloud SQL instance or add more read replicas without impacting the compute services.

- **Asynchronous Processing for Throughput:** The heavy reliance on a Pub/Sub event bus for inter-service communication is a key scaling pattern. It allows the platform to absorb massive bursts of work (e.g., a bulk upload of 10,000 documents) by queuing the tasks. The downstream services can then process these tasks at their own maximum sustainable rate, ensuring system stability.

## 11.2. Cost Governance and Optimization

- **Token Usage Monitoring:** The most significant variable cost is LLM token consumption. The platform will have a centralized logging mechanism that records the token count (input and output) for every single LLM call, tagged with the project\_id and the agent\_name. This data will be exported to BigQuery for detailed analysis.
- **Project-Based Quotas & Budgets:** We will build a simple dashboard on top of this BigQuery data that allows Platform Operators to monitor token usage per engagement project. We will implement soft quotas and automated budget alerts to prevent runaway costs.
- **Model Tiering:** The Inference Abstraction Layer will be designed to support model tiering. For less complex tasks (e.g., simple text extraction or classification), the system can be configured to use a smaller, faster, and cheaper model (e.g., Gemini Flash), while reserving the most powerful and expensive models (e.g., Gemini 1.5 Pro) for complex reasoning and proposal generation, optimizing the cost-performance trade-off.
- **Automated Budget Enforcement:** To provide hard guardrails against cost overruns, we will implement an automated enforcement mechanism.
  - **Mechanism:** A scheduled **Cloud Function** will run periodically (e.g., every hour) to query the token usage data in **BigQuery**. If a project's usage exceeds a pre-defined budget threshold (e.g., 80%), this function will call a configuration endpoint on the Reasoning Service to either:
    - **Throttle** future requests for that project.
    - Automatically switch the project to a "**low-cost mode**," forcing the use of cheaper models like Gemini Flash for all subsequent tasks.
    - Trigger a high-priority alert to the Platform Operator and the project owner.

This enforcement is governed by a 'Cost Runbook' that explicitly maps task types to token budgets and defines the fallback behavior (e.g., switch to cheaper model, queue for off-peak processing) when a threshold is breached.

This approach to cost monitoring, optimization, and governance aligns with the principles of the FinOps Foundation's Cloud Financial Management framework.

| <i><b>Task Category</b></i>                                                                     | <b>Primary Model (Default)</b> | <b>Fallback/Cost-Optimized Model</b> | <b>Rationale</b>                                                      |
|-------------------------------------------------------------------------------------------------|--------------------------------|--------------------------------------|-----------------------------------------------------------------------|
| <b><i>Complex Reasoning &amp; Synthesis</i></b> (e.g., <i>Final Proposal Generation</i> )       | Gemini 2.5 Pro                 | Gemini 2.5 Flash                     | Prioritizes highest quality output for critical, user-facing tasks.   |
| <b><i>Fact Extraction &amp; Classification</i></b> (e.g., <i>Document Processing Pipeline</i> ) | Gemini 2.5 Pro                 | Gemini 2.5 Flash                     | Optimizes for speed and cost on high-volume, structured output tasks. |
| <b><i>Interactive Chat (Low Latency)</i></b>                                                    | Gemini 2.5 Flash               | Gemini 2.5 Pro                       | Prioritizes fast response times for a fluid user experience.          |

### 11.3 Unified Observability Context

To provide end-to-end traceability for complex, asynchronous agentic workflows, the platform enforces a unified observability context that is propagated across all services, events, and API calls.

- **The Correlation Context:** Every top-level request (e.g., an API call to generate a document) initiates a **Correlation Context**. This context is a small JSON object that contains, at a minimum:
  - `correlationId`: A unique ID for the entire end-to-end transaction (e.g., a UUID).
  - `jobId`: The ID of the specific workflow or task being executed.
  - `projectId`: The ID of the client engagement project.
- **Propagation Mechanism:**
  1. **API Gateway:** The API Gateway is responsible for generating the initial Correlation Context for all incoming user requests. It injects this context as an HTTP header.

2. **Internal Services:** All internal services (Cloud Run, GKE) are required to extract this header and use it to enrich their structured logs.
3. **Pub/Sub Events:** When publishing an event to Pub/Sub, services must include the Correlation Context as a message attribute. The consuming service is then responsible for extracting it and continuing the trace.
4. **Agent & LLM Calls:** The context is passed through CrewAI, AutoGen, and Semantic Kernel. The Inference Abstraction Layer is responsible for injecting the correlationId into the metadata of every LLM call.
- **Outcome:** This discipline ensures that a single correlationId can be used in **Cloud Logging** and **Cloud Trace** to retrieve a complete, ordered view of an entire workflow, from the initial user click, through multiple Pub/Sub events and agent collaborations, to the final LLM call and database write.
  - **Log Enrichment:** Crucially, the correlationId is automatically injected into every structured log entry generated by any service or agent involved in the transaction. This enables powerful, request-scoped log filtering in Cloud Logging.
  - Specific operational dashboards will be created for key processes, including a 'Fusion Health' dashboard tracking merge rate, human review queue length, and average confidence scores to monitor for model drift.
  - **Log Enrichment:** Crucially, the correlationId is automatically injected into every structured log entry generated by any service or agent involved in the transaction. This enables powerful, request-scoped log filtering in Cloud Logging.

---

## 12. Phased Rollout & Immediate Next Steps

The platform will be developed iteratively to de-risk assumptions and deliver value quickly. The following plan outlines the initial release phases and their measurable outcomes.

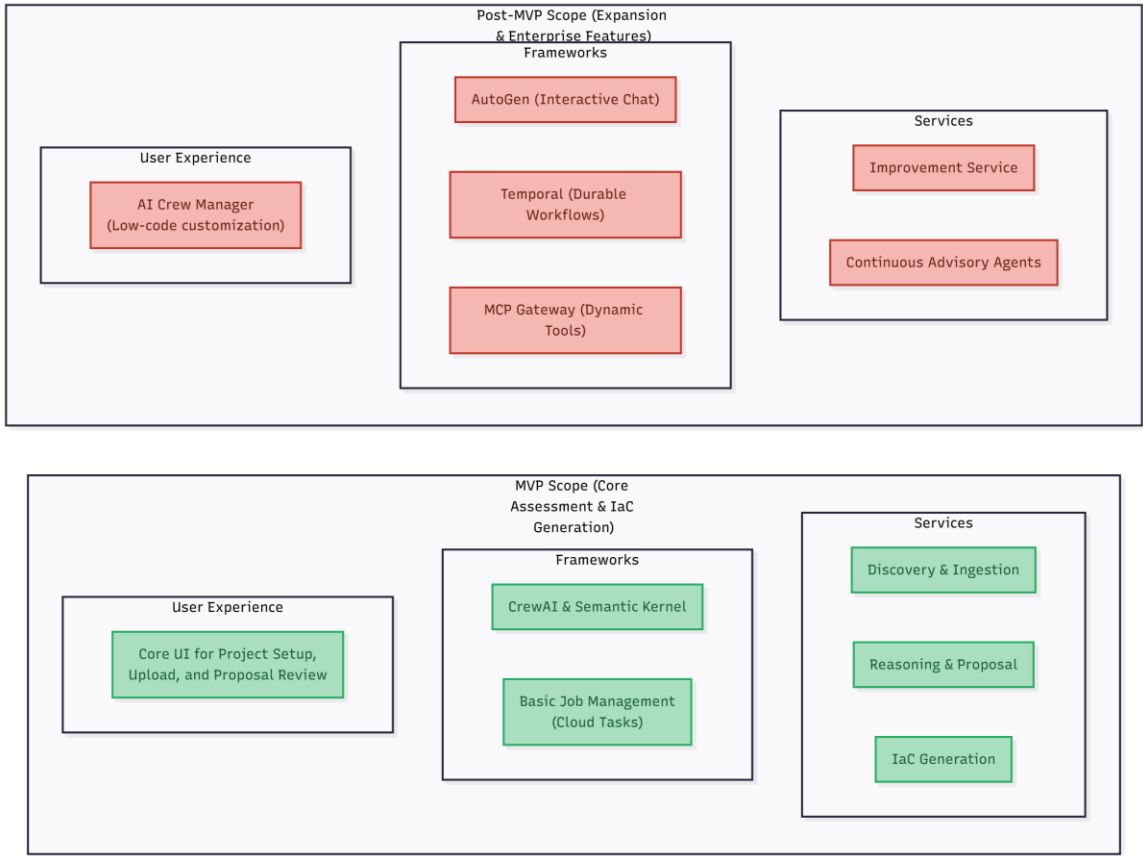
| PHASE   | CAPABILITY               | DELIVERY TARGET | MEASURE OF DONE                                                                                      |
|---------|--------------------------|-----------------|------------------------------------------------------------------------------------------------------|
| SPIKE 1 | Chat & AutoGen Precision | 6 weeks         | Interactive chat PoC is functional; RAG pipeline achieves precision@5 $\geq 0.80$ on golden dataset. |
| SPIKE 2 | End-to-End Tracer Bullet | 10 weeks        | A single document can be processed to generate a valid IaC pull request. MVP workflow validated.     |



|                              |                                   |          |                                                                                                                          |
|------------------------------|-----------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------|
| <b>SPIKE 3</b>               | Temporal & Durability Integration | 14 weeks | A long-running mock workflow can be successfully started, paused, and resumed via Temporal. Reliable recovery validated. |
| <b>PHASE 1 (MVP RELEASE)</b> | Core Assessment & IaC Platform    | 18 weeks | Meets all Measurable MVP Acceptance Criteria (Section 18.5) in pilot engagements.                                        |

### The MVP: Functional Cut Line

While the blueprint describes the full vision, the initial Minimum Viable Product (MVP) will focus on delivering the core value chain for a single, high-impact use case: **Automated Infrastructure Assessment and IaC Generation for VM Migration**. The following table defines the "cut line" between MVP and future phases.

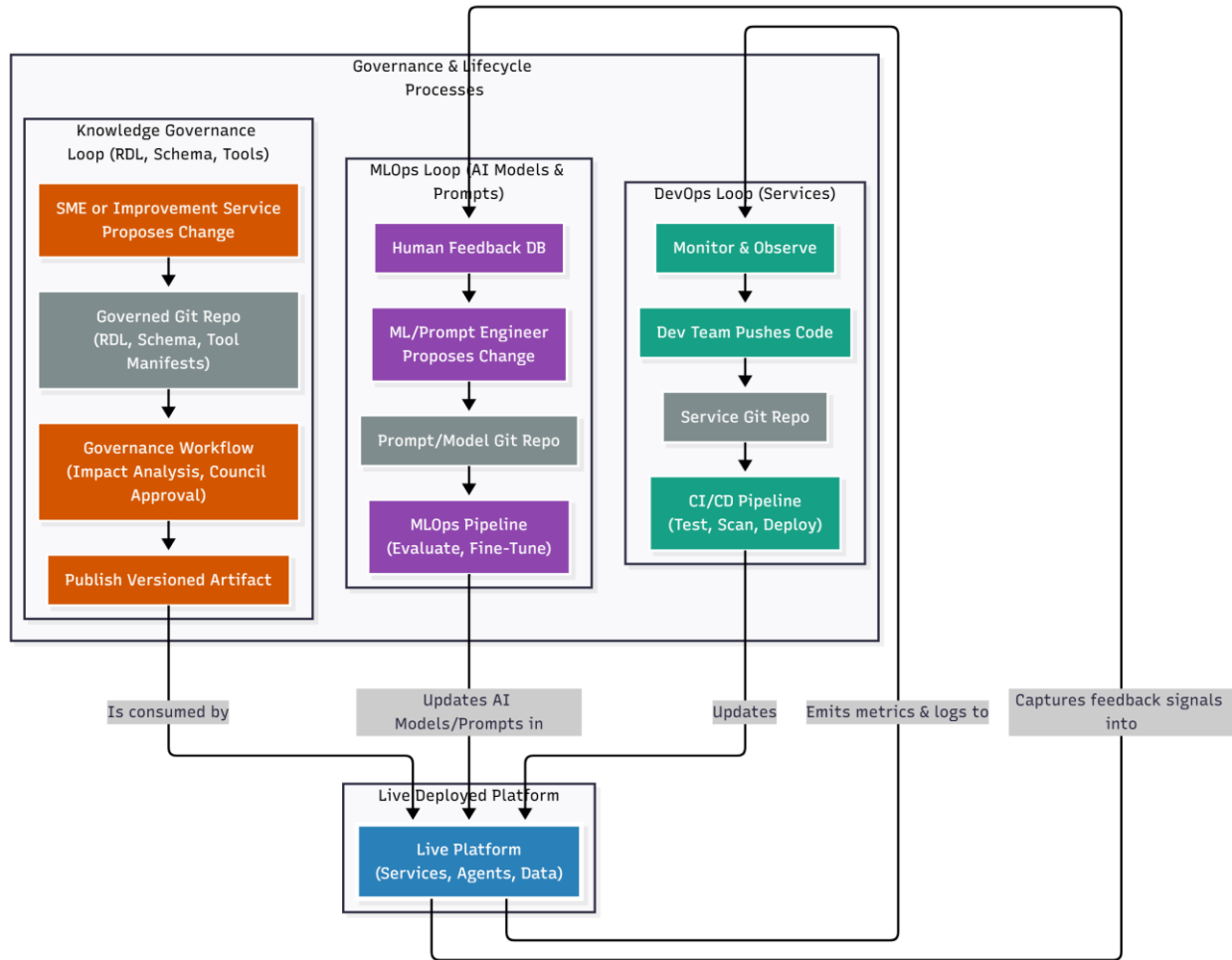


| ARCHITECTURAL LAYER / CAPABILITY | MVP (MUST-HAVE)                      | POST-MVP (FUTURE PHASES)                        |
|----------------------------------|--------------------------------------|-------------------------------------------------|
| CORE SERVICES                    | Discovery, Reasoning, IaC Generation | Improvement Service, Continuous Advisory Agents |

|                       |                                                                               |                                                          |
|-----------------------|-------------------------------------------------------------------------------|----------------------------------------------------------|
| AGENT FRAMEWORKS      | <b>CrewAI</b> (for deterministic assessment workflow), <b>Semantic Kernel</b> | <b>AutoGen</b> (interactive chat is a follow-on feature) |
| WORKFLOW ENGINE       | Basic job management (Cloud Tasks)                                            | <b>Temporal</b> (for full durable execution in Phase 2)  |
| KNOWLEDGE LAYER       | GCS, Cloud SQL, Neo4j, Vector DB, Elasticsearch                               | Schema Migration Tooling                                 |
| TOOLING & INTEGRATION | Hardcoded agent tools for RAG & Graph                                         | <b>MCP Gateway</b> (dynamic tool loading)                |
| GOVERNANCE            | Manual PR reviews for RDL & Prompts                                           | RDL Governing Council, Automated Governance Workflows    |
| USER EXPERIENCE       | Core UI for project setup, document upload, and proposal review               | Full AI Crew Manager & "low-code" customization          |

This phased approach ensures the initial product is focused, delivers value quickly, and validates the core architecture before expanding to the full feature set.

### 13. Governance Model for the Reference Decision Library (RDL)



*Lifecycle and Governance diagram*

The RDL is the codified expertise of our organization and the logical core of the platform. Its integrity, consistency, and management are critical to the platform's success and trustworthiness. A formal governance model is therefore essential.

### 13.1. Principles of RDL Governance

- Federated Ownership:** The RDL is not a monolithic entity owned by a single team. Ownership will be federated. The Cloud Center of Excellence (CCoE) will own the core policies (e.g., security, networking baselines), while individual application domains or business units can own the decision templates relevant to their specific technology stacks (e.g., the Java CoE owns the templates for Tomcat workloads).

- **Human Oversight is Non-Negotiable:** Every change to the RDL, whether proposed by a human or the Improvement Service, must be reviewed and approved by a human owner via a pull request.
- **Transparency and Versioning:** All aspects of the RDL (policies, templates, playbooks) will be stored in a version-controlled Git repository. The history of every change, including who approved it and why, will be transparent and auditable.

## 13.2. RDL Governance Roles & Responsibilities

To ensure consistency without introducing excessive bureaucracy, RDL governance will be managed through a lightweight, role-based model rather than a formal council.

- **RDL Maintainers:** A small, designated group of senior architects will be assigned as the official "Maintainers" of the rdl repository (enforced via the CODEOWNERS file).
- **Responsibilities of Maintainers:**
  - **Conflict Resolution:** They are responsible for adjudicating conflicting rules proposed by different teams or agents.
  - **Core Policy Ownership:** They have the final approval authority for all changes to enterprise-wide core policies (e.g., security, compliance).
  - **Lifecycle Management:** They guide the process for introducing, deprecating, and retiring decision templates and policies.
- **Review SLA:** For non-critical changes, Maintainers are expected to provide a review within **48 business hours**. For critical bug fixes, an expedited review process is available.
- **Emergency Rollback:** In the event a merged RDL change causes significant regressions in proposal quality, any Maintainer can trigger an emergency rollback. This involves an automated script that immediately redeploys the last known good, versioned RDL artifact and freezes the main branch of the rdl repository until a root cause analysis is complete.

## 13.3. The RDL Change Management Workflow

1. **Proposal:** A change is proposed by either a human architect or the Improvement Service creating a pull request in the rdl Git repository.
2. **Automated Validation (CI):** A CI pipeline automatically runs, performing:
  - **Linting:** Validating the syntax of Rego policies and YAML templates.

- **Unit Testing:** Running pre-defined unit tests for the policy or template.
  - **Impact Analysis:** An automated script runs the proposed change against a golden dataset of workloads to simulate its impact and flags any unexpected or widespread changes in recommendations.
  - **Fusion Accuracy Test:** For changes affecting the fusion logic, the pipeline runs against a labeled dataset of entity pairs to calculate precision and recall, preventing regressions in merge accuracy
3. **Human Review & Approval:** The PR is assigned to the designated owners based on the CODEOWNERS file in the repository. The owners review the change and the results of the impact analysis.
  4. **Merge & Publish:** Upon approval and merge, a CD pipeline automatically versions and publishes the RDL as a new immutable artifact (e.g., a versioned container image or GCS object) that is consumed by the platform's services.

## 13.4. Knowledge Schema Versioning and Migration Strategy

The Knowledge Graph's schema (the types of nodes, properties, and relationships) is a core part of the platform's data contract. Like the RDL, it must be managed with a formal governance and versioning strategy to prevent breaking changes.

- **Schema as Code:** The canonical Neo4j graph schema will be defined as code in a dedicated Git repository (knowledge-schema). This includes definitions for node labels, properties, and relationship types.
- **Governance:** Changes to the schema require a pull request and are subject to review by the **RDL Maintainers** to ensure the proposed changes are aligned with the platform's data model.
- **Migration Mechanism:** We will use a database migration tool designed for Neo4j (similar to Liquibase or Flyway for relational databases).
  1. When a schema change is approved, a developer will write a corresponding **migration script** (e.g., a Cypher script) that safely alters the existing graph data to conform to the new schema (e.g., RENAME property, MERGE nodes).
  2. This migration script is versioned and committed to the knowledge-schema repository.

3. During deployment, the CI/CD pipeline will automatically apply any new, un-applied migration scripts to the Neo4j database before the new application code is deployed, ensuring the data and application are always in sync.
4. For long-running migrations that affect large portions of the graph, the migration script can be wrapped in a Temporal workflow. This ensures the migration runs durably, with retries and observability, preventing the database from being left in an inconsistent state due to a partial failure.

## 14. Operational Mode: Continuous Advisory

To fulfill the "Phase 4: Operate & Evolve" vision of the Transformation Framework, the platform is designed to transition from a project-based tool into a persistent, long-running advisory product for the client. This operational mode focuses on continuous monitoring, optimization, and governance.

### 14.1. Live Data Adapters

The Discovery & Ingestion Service will be equipped with a suite of continuously running adapters that stream live operational data from the client's cloud environment into the Knowledge Graph, ensuring it remains an up-to-date "digital twin." These include connectors for:

- **Cloud Financial Management:** AWS Cost and Usage Report (CUR), Azure Cost Management APIs, Google Cloud Billing exports.
- **Cloud Observability:** AWS CloudWatch, Azure Monitor, and Google Cloud Monitoring for key performance and reliability metrics.
- **Cloud Security Posture:** AWS Security Hub, Microsoft Defender for Cloud, and Google Security Command Center for live compliance and security risk data.

This operational mode introduces a new class of **Advisory Agents** that are not triggered by direct user requests, but run on a schedule or in response to live events from the data streams.

- **Guardrail Agent:** Continuously monitors live spend against budgets and triggers real-time anomaly alerts for "cost shock" prevention.

- **SRE Agent:** Monitors performance and reliability dashboards, automatically creating tickets or notifications when SLOs are at risk and suggesting reliability improvements.
- **Maturity Assessor Agent:** A scheduled agent that runs quarterly to re-assess the client's operational posture against a cloud maturity model, generating an updated Cloud Maturity Scorecard and Risk & Compliance Register.

## 15. Incident Response & Operational Runbooks

The platform is designed with security guardrails, but a formal incident response plan is essential for handling unforeseen events.

### 15.1. Incident Categories & Automated Containment

- **Category 1: Potential Data Exfiltration:**
  - **Trigger:** Cloud IDS (Intrusion Detection System) alert on anomalous egress traffic.
  - **Automated Response:** An automated workflow (e.g., a Cloud Function triggered by the alert) will immediately **revoke the IAM credentials** of the suspected service account and apply a firewall rule to **deny all egress** from the compromised component.
- **Category 2: Malicious or Corrupted RDL/Prompt:**
  - **Trigger:** Monitoring alert for a sudden, drastic change in proposal acceptance rates or multiple OPA validation failures.
  - **Automated Response:** The platform will automatically initiate a **"safe mode" rollback**, reconfiguring the Reasoning Service to use the last known good, versioned RDL/Prompt artifact.

### 15.2. Forensic & Recovery Process

1. **Forensic Snapshot:** Upon containment, a forensic snapshot of the compromised service's state and relevant logs is taken for analysis.
2. **Key Rotation:** The incident response team will initiate a manual rotation of all relevant secrets and keys in **Secret Manager** and **Cloud KMS**.

3. **Triage & Remediation:** The incident commander, guided by a roles and responsibilities matrix, will lead the investigation and remediation.
4. **Post-Mortem:** A blameless post-mortem will be conducted to identify the root cause and implement preventative measures.

A detailed runbook will be maintained in the platform-infra repository, specifying the exact gcloud/kubectl commands and the roles (e.g., 'On-Call SRE', 'Security Lead') responsible for executing each step of the containment and recovery process.

### 15.3. Failure Mode Examples & Recovery Stories

The platform's resilience is demonstrated by its automated recovery from common failure modes.

- **Story 1: A Cloud Run instance fails mid-fusion.**
  - **Detection:** The Temporal worker running the LLM comparison activity fails to send a heartbeat.
  - **Recovery:** **Temporal** automatically detects the timeout, preserves the activity's state (the input data), and re-schedules it on a different, healthy Cloud Run instance. The workflow continues seamlessly with no data loss.
- **Story 2: The primary LLM (Gemini Pro) becomes unavailable.**
  - **Detection:** The Inference Abstraction Layer detects a series of failed API calls or a latency spike exceeding the SLO.
  - **Recovery:** The **Inference Abstraction Layer** automatically triggers its fallback strategy, rerouting all subsequent LLM calls to the pre-configured secondary model (e.g., Gemini Flash). A high-priority alert is sent to the Platform Operator.
- **Story 3: A Knowledge Graph ingestion job is poisoned by a malformed document.**
  - **Detection:** The Discovery & Ingestion Service job fails with a parsing error.
  - **Recovery:** The job fails, but the original EvidenceBundle is retained in GCS. The failure event is logged with the correlationId. A Platform Operator is notified, can inspect the problematic document, remove it from the bundle, and re-trigger the ingestion job for the remaining documents.



A detailed runbook will be maintained in the platform-infra repository, specifying the exact gcloud/kubectl commands and the roles (e.g., 'On-Call SRE', 'Security Lead') responsible for executing each step of the containment and recovery process.

## 16. Appendices

### 16.1. Core Event Schema Examples (JSON Schema)

#### DiscoveryEvent:

```
1. codeJSON
2. {
3.   "$schema": "http://json-schema.org/draft-07/schema#",
4.   "title": "DiscoveryEvent",
5.   "description": "Fired when a new evidence bundle is collected.",
6.   "type": "object",
7.   "properties": {
8.     "eventId": {"type": "string", "format": "uuid"},
9.     "timestamp": {"type": "string", "format": "date-time"},
10.    "projectId": {"type": "string"},
11.    "source": {"type": "string", "enum": ["vcenter", "aws", "azure", "manual_upload"]},
12.    "payload": {
13.      "type": "object",
14.      "properties": {
15.        "bundleGcsPath": {"type": "string"},
16.        "bundleChecksum": {"type": "string"}
17.      },
18.      "required": ["bundleGcsPath", "bundleChecksum"]
19.    }
20.  },
21.  "required": ["eventId", "timestamp", "projectId", "source", "payload"]
22. }
23. (...similar concrete JSON Schema examples for ProposalReady and ProposalCorrected would follow...)
24.
```

### 16.2. Minimum Audit Log Schema

All critical events (especially LLM calls and tool usage) will be logged with the following minimum fields for reconstruction and compliance:

- timestamp: ISO8601 UTC timestamp.
- correlationId: End-to-end request tracing ID.
- projectId: The engagement project ID.
- actor: The identity initiating the action (e.g., user:[email] or service:[service\_name]).
- agentName: The specific agent performing the action.

- `actionType`: e.g., `LLM_CALL`, `TOOL_CALL`.
- `promptHash`: SHA256 hash of the prompt sent to the LLM (to avoid logging sensitive data).
- `modelVersion`: The specific version of the LLM used.
- `tokenCount`: { "input": number, "output": number }.
- `evidenceIds`: Array of foundational fact IDs used as context.
- `decisionId`: Link to the resulting Proposal or other generated artifact.
- `outcome`: `SUCCESS` or `FAILURE`.

### 16.3. RDL & Prompt Versioning Strategy

Versioning is critical for reproducibility and governance.

- **Semantic Versioning:** All Git repositories (`rdl`, `prompt-library`) will use MAJOR.MINOR.PATCH semantic versioning. A MAJOR version change indicates a breaking change in logic or output schema.
- **Immutable Artifacts:** The CD pipeline for these repos will not just update a latest tag. It will build and publish an immutable, version-tagged artifact (e.g., a container image `rdl-rules:1.2.0`). The platform services are then configured to consume a specific version, allowing for safe, controlled rollouts and instant rollbacks. Artifacts and their provenance will be retained according to a defined data retention policy (e.g., 7 years for audit purposes).

### 16.4. IaC Generation & Pre-Merge Validation

To prevent flawed IaC from ever reaching a main branch, the workflow of the **Generation & Execution Service** is enhanced:

1. **Generate IaC:** The service orchestrates an agent to generate the Terraform code.
2. **Provision Sandbox:** The CI pipeline (triggered on the PR) automatically provisions a temporary, isolated GCP project to act as a "preview sandbox."
3. **Dry-Run (terraform apply):** The pipeline runs `terraform apply` against this sandbox environment.
4. **Run Smoke Tests:** After the apply succeeds, a basic set of automated smoke tests are run against the newly created resources.

5. **Report & Destroy:** The results of the apply and smoke tests are posted as a comment to the PR. The sandbox environment is then automatically destroyed. A PR cannot be merged unless this check passes.

## 16.5. Measurable MVP Acceptance Criteria

The initial MVP will be considered successful upon meeting the following concrete criteria in pilot engagements:

- **RAG Precision:** The retrieval pipeline achieves a **precision@5 of  $\geq 0.80$**  on the pre-defined golden dataset.
- **Proposal Acceptance Rate:** **>60%** of generated proposals for a core use case (e.g., VM rightsizing) are accepted by senior architects (including minor edits).
- **Ingestion SLO:** **99%** of evidence bundles are ingested and visible in Neo4j within **6 hours** for pilot engagements.
- **Safety & Compliance:** **100%** of IaC PRs containing known destructive or non-compliant configurations (as defined in a test suite) are successfully **blocked by the OPA validation step**.
- **Egress Control:** During an on-prem pilot, network logs must show **zero connections** originating from the on-prem Discovery Agent Gateway to any IP address outside the pre-approved allowlist.

## 16.6. MCP Tool & Token Schema Examples

Sample `capability_manifest.json`:

```
1. codeJSON
2. {
3.   "manifestVersion": "1.0",
4.   "toolName": "aws-pricing-tool",
5.   "description": "Fetches live pricing for AWS services.",
6.   "functions": [
7.     {
8.       "name": "get_ec2_price",
9.       "description": "Get the on-demand price for an EC2 instance type.",
10.      "parameters": {
11.        "type": "object",
12.        "properties": {
13.          "instanceType": {"type": "string"},
14.          "region": {"type": "string"}
15.        },
16.        "required": ["instanceType", "region"]
17.      },
18.      "scopeRequired": "aws:pricing:read"
19.    }
20.  ]
21. }
```

**Sample capability\_token JWT Payload:**

```
1. codeJSON
2. {
3.   "iss": "AgenticModernizationPlatform",
4.   "sub": "reasoning-service",
5.   "aud": "mcp-gateway",
6.   "job_id": "job-uuid-1234",
7.   "scopes": [
8.     "aws:pricing:read",
9.     "gcp:compute:list"
10.  ],
11.   "exp": 1678886400,
12.   "iat": 1678882800
13. }
14.
```

## 16.7 Discovery Permissions and Service Identities

To perform read-only discovery, the platform requires a set of minimal, least-privilege IAM roles/permissions in the client's source environment. The following are examples:

**For VMware vSphere:**

- **Identity:** A single Service Account user in vCenter.
- **Role:** Read-only role assigned at the root vCenter level.

**For AWS:**

- **Identity:** An IAM Role in the target AWS account that the platform's service account can assume.
- **Minimal Policies:**
  - AWSMigrationHubReadOnly
  - AmazonEC2ReadOnlyAccess
  - AmazonS3ReadOnlyAccess
  - ReadOnlyAccess for AWS Application Discovery Service.

**For Azure:**

- **Identity:** An App Registration (Service Principal) in the client's Azure AD tenant.
- **Minimal Role:** Reader role assigned at the subscription or management group scope.

# 17. Architect's Index

This index provides quick navigation to key architectural decision areas.

- **Security & Governance**
  - Network Security & Hybrid Topology: 9.1, 9.1.b
  - Supply Chain Security (SLSA): 9.2
  - AI Safety & Guardrails: 9.6
  - Incident Response Plan: 15.0
- **AI & Agent Architecture**
  - Layered Technology Stack: 5.1
  - Runtime Interaction Model (Sequence Diagram): 5.4
  - Agent Inventory: 4.2
  - Prompt Lifecycle Management: 9.3
  - Model Serving & Governance: 9.7
- **Data & Knowledge Management**
  - Document Processing & Fusion Pipeline: Phase B (in Section 7)
  - Knowledge Schema Governance: 13.4
  - Fusion Scalability Model: 9.4
- **Operations & Lifecycle**
  - CI/CD & Polyrepo Structure: 10.0
  - RDL Governance: 13.0
  - Tool Certification Workflow: 5.5
  - MVP Cut Line: The MVP (in Section 12)

## 18. Future Considerations: Federated Control Plane

While the current architecture focuses on single-tenant client deployments, the long-term vision includes managing multiple, independent tenant instances from a centralized control plane. To prepare for this future state, a placeholder for a **Federated Control Plane** is considered in the design.

- **Purpose:** A lightweight, central service, managed by our organization, to handle license management, push version updates, monitor the health of all tenant deployments, and aggregate anonymized usage telemetry.
- **Impact:** This future-proofs the design for scalability, enabling us to manage a fleet of client deployments efficiently without requiring a fundamental shift to a multi-tenant architecture. The specific architecture of this control plane is deferred but will be guided by the principles of minimal privilege and strict tenant isolation.